

Assigment2

Insudtrial Control

Department of Electrical Engineering

Student: Hamed Batani 402101339
Instructor: Prof. Ahi

---

1 Problem 1

In this problem, we are tasked with tuning PID controllers for two given transfer functions, $G_1(s)$ and $G_2(s)$, using the Chien-Hrones-Reswick (CHR) method for setpoint tracking. The objective is to design and compare controllers for both "0% overshoot" and "20% overshoot" criteria.

The implementation in MATLAB follows a structured numerical approach to apply the CHR method:

1. **System Definition:** The transfer functions $G_1(s)$ and $G_2(s)$ are defined using MATLAB's `tf` function.
2. **Open-Loop Step Response and Inflection Point:** The CHR method requires finding the tangent line at the inflection point (maximum slope) of the open-loop step response. We use the `step()` function to simulate the open-loop response. The numerical derivative is calculated using `diff(y) ./ diff(t)` to find the maximum slope (m) and its corresponding time (t_{inf}) and amplitude (y_{inf}).
3. **Calculating a and L:** Based on the geometric properties of the tangent line $y - y_{inf} = m(t - t_{inf})$, we calculate the CHR parameters:
   - L is the x-intercept (delay time), calculated as $L = t_{inf} - \frac{y_{inf}}{m}$.
   - a is the absolute y-intercept, calculated as $a = |m(0 - t_{inf}) + y_{inf}|$.
4. **CHR Tuning Rules:** Using the calculated a and L, the PID parameters (K_c, T_i, T_d) are determined using the standard CHR setpoint tracking tables for both "no overshoot" and "20% overshoot" targets.
5. **PID Controller Construction:** The PID controller is implemented using MATLAB's `pid()` function. To ensure physical realizability and noise attenuation, a derivative filter is included using $N = 10$. The standard PID form $C(s) = K_c(1 + \frac{1}{T_i s} + \frac{T_d s}{1 + T_d s/N})$ is mapped to MATLAB's parallel form arguments.

6. **Closed-Loop Analysis:** The closed-loop system is formed using unity negative feedback via `feedback(C * G, 1)`. Finally, the closed-loop step responses are plotted, and the actual overshoot is extracted using `stepinfo()`.

```
clear; clc; close all;

s = tf('s');
G1 = (80 / (s + 8)^2) * (1 / (s + 2) + 0.4 / (s + 0.04));
G2 = (s + 5)^2 / (s * (s + 2)^2 * (s + 25));

systems = {G1, G2};
sys_names = {'G_1(s)', 'G_2(s)'};
N = 10;

figure('Name', 'CHR PID Tuning Results', 'Position', [100, 100, 1000, 400]);

for i = 1:length(systems)
    G = systems{i};

    [y, t] = step(G);

    dy = diff(y) ./ diff(t);
    [max_slope, max_idx] = max(dy);

    t_inf = t(max_idx);
    y_inf = y(max_idx);

    L = t_inf - y_inf / max_slope;
    a = abs(max_slope * (0 - t_inf) + y_inf);

    K_no = 0.6 / a;
    Ti_no = 1.0 * L;
    Td_no = 0.5 * L;

    K_20 = 0.95 / a;
    Ti_20 = 1.357 * L;
    Td_20 = 0.473 * L;

    C_no = pid(K_no, K_no/Ti_no, K_no*Td_no, Td_no/N);
    C_20 = pid(K_20, K_20/Ti_20, K_20*Td_20, Td_20/N);

    sys_cl_no = feedback(C_no * G, 1);
    sys_cl_20 = feedback(C_20 * G, 1);

    subplot(1, 2, i);
    [y_no, t_no] = step(sys_cl_no);
    [y_20, t_20] = step(sys_cl_20);
```

```
    plot(t_no, y_no, 'g', 'LineWidth', 2); hold on;
    plot(t_20, y_20, 'm', 'LineWidth', 2);

    grid on;
    title(['Closed-Loop Step Responses for ', sys_names{i}]);
    xlabel('Time (s)'); ylabel('Amplitude');
    legend('No Overshoot Tracking', '20% Overshoot Tracking', 'Location', 'Best');

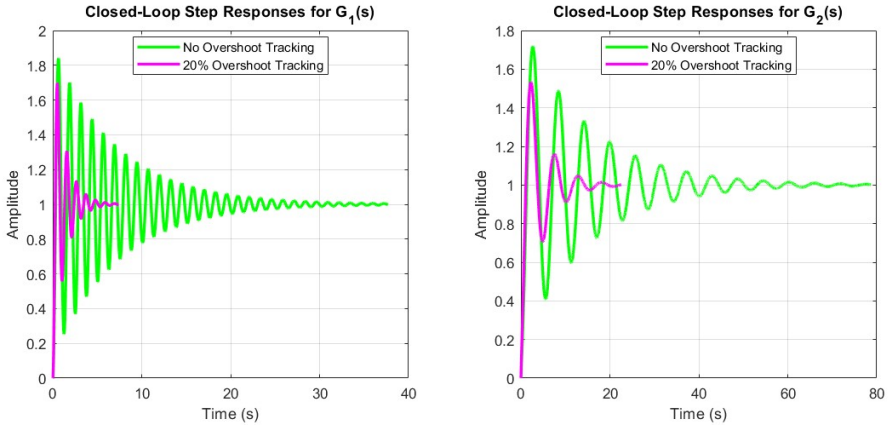
    info_no = stepinfo(sys_cl_no);
    info_20 = stepinfo(sys_cl_20);

    fprintf('\nPerformance for %s\n', sys_names{i});
    fprintf('No Overshoot:\n  Overshoot: %.2f%%\n', info_no.Overshoot);
    fprintf('20%% Overshoot:\n  Overshoot: %.2f%%\n', info_20.Overshoot);
end
```

Command Window Output

```
Performance for G_1(s)
No Overshoot:
  Overshoot: 84.10%
20% Overshoot:
  Overshoot: 69.65%

Performance for G_2(s)
No Overshoot:
  Overshoot: 71.82%
20% Overshoot:
  Overshoot: 53.44%
>>
```



2 System Modeling and Parameter Estimation

The objective of this analysis is to approximate a high-order linear time-invariant system, specifically $G(s) = \frac{1}{(s+1)^8}$, using lower-order equivalent models. We aim to estimate the parameters for three distinct model configurations. The original system is simulated over a 40-second horizon using a discrete time step dt. The steady-state process gain k is extracted from the final value of the step response, yielding $k = 1.0$.

2.1 Part (a): First-Order Model (Residence Time Approximation)

The first-order system is described by the transfer function $G_a(s) = \frac{k}{T_a s+1}$. The residence time approximation method utilizes the integral of the difference between the steady-state response and the transient response, defined as the area $A_0 = \int (k - y(t))dt$. The time constant T_a is strictly defined as the average residence time, computed as $T_a = \frac{A_0}{k}$. In MATLAB, A_0 is approximated using a Riemann sum.

Based on the computed parameters, the resulting model is:

$$G_a(s) = \frac{1}{8.200000s + 1}$$

2.2 Part (b): First-Order Plus Dead Time Model (Area Method)

This three-parameter model is defined as $G_b(s) = \frac{k}{T_b s+1} e^{-L_b s}$. Using the Area Method, we establish an integration limit $T_{ar} = \frac{A_0}{k}$. We then calculate the area $A_1 = \int_0^{T_{ar}} y(t)dt$ up to this point. The time constant T_b is derived as $T_b = e\frac{A_1}{k}$, and the dead time L_b is the difference $L_b = T_{ar} - T_b$. In MATLAB, 'find' is used to bound the integration array for A_1, followed by numerical summation.

Based on the computed parameters, the resulting model is:

$$G_b(s) = \frac{1}{3.337946s + 1} e^{-4.862054s}$$

2.3 Part (c): Second-Order Plus Dead Time Model (Four-Parameter Model)

The higher-fidelity four-parameter model is defined as $G_c(s) = \frac{k}{(T_1 s+1)(T_2 s+1)} e^{-L_c s}$. The dead time L_c is approximated by finding the maximum rate of change (inflection point) of the step response curve and determining the x-intercept of the tangent line at this point. To resolve the two time constants T_1 and T_2, the model equation is matched to the actual response at two distinct response thresholds, specifically at 20% and 80% of the steady-state value. A system of nonlinear equations is formulated and resolved numerically using MATLAB's 'fsolve' optimization routine.

Based on the computed parameters, the resulting model is:

$$G_c(s) = \frac{1}{(3.399607s + 1)(0.570361s + 1)} e^{-4.304286s}$$

The step responses of the original eighth-order system and the three derived approximations are visual-

ized in Figure below

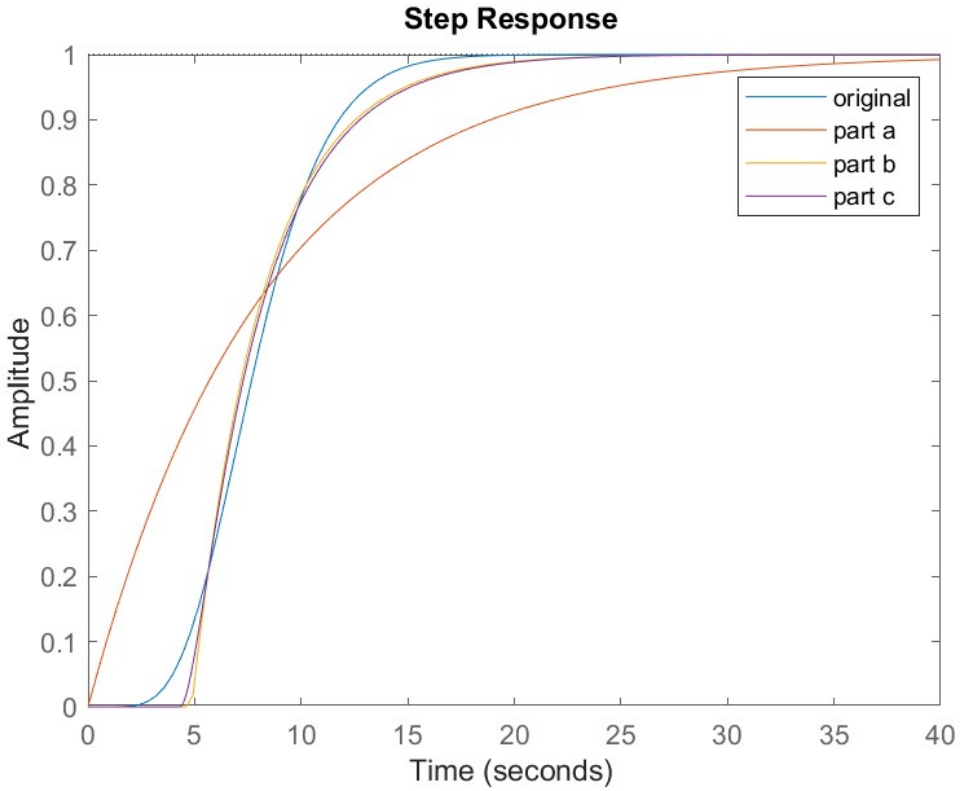


Figure 1: Comparison of step responses for the original system and the derived reduced-order models.

As observed, the simple first-order approximation (Part a) fails to capture the significant initial latency (S-shape curve) of the eighth-order system. The inclusion of dead time in Part (b) vastly improves the alignment of the transient phase. Part (c), which utilizes a second-order polynomial alongside dead time, provides the most accurate morphological match to the original system's step response.

Command Window Output:

```
>> Q2
part a:
k: 1.000000
t_a: 8.200000

part b:
k: 1.000000
t_b: 3.337946
l_b: 4.862054

part c:
k: 1.000000
t1_c: 3.399607
t2_c: 0.570361
```

```
l_c: 4.304286
```

MATLAB Script:

```
s = tf('s');
g = 1/(s+1)^8 ;
[y, t] = step(g, 40);
dt = t(2) - t(1) ;
k = y(end);

% part a ( residence time approximation method was used here )
a0 = sum(k - y) * dt;
t_a = a0 / k;

g_a = tf(k, [t_a 1]) ;

fprintf('part a:\n')
fprintf('k: %f\n', k)
fprintf('t_a: %f\n', t_a)

% part b ( area method )
tar = a0 / k ;
idx = find(t <= tar, 1, 'last');
a1 = sum(y(1:idx)) * dt;

t_b = (exp(1) * a1) /k;
l_b = tar - t_b;

g_b = tf(k, [t_b 1], 'ioDelay', l_b);

fprintf('\npart b:\n')
fprintf('k: %f\n', k)
fprintf('t_b: %f\n', t_b)
fprintf('l_b: %f\n', l_b)

% part c ( four parameter model )
dydt = diff(y) ./ diff(t);
[m, im] = max(dydt);
tm = t(im);
ym = y(im);

l_c = tm - ym/m;
a_val = abs(ym - m*tm);

% as disscused in class , 0.2 and 0.8 would be good time to match
idx_20 = find(y >= 0.2*k, 1);
t20 = t(idx_20);
idx_80 = find(y >= 0.8*k, 1);
```

```
t80 = t(idx_80);

% numerical solve for t1 and t2
fun = @(x) [1 - (x(1)*exp(-(t20-l_c)/x(1)) - x(2)*exp(-(t20-l_c)/x(2)))/(x(1)-
           1 - (x(1)*exp(-(t80-l_c)/x(1)) - x(2)*exp(-(t80-l_c)/x(2)))/(x(1)-

x0 = [2; 1];
opts = optimoptions('fsolve', 'Display', 'off');
sol = fsolve(fun, x0, opts);

t1_c = sol(1);
t2_c = sol(2);

fprintf('\npart c:\n')
fprintf('k: %f\n', k)
fprintf('t1_c: %f\n', t1_c)
fprintf('t2_c: %f\n', t2_c)
fprintf('l_c: %f\n', l_c)
g_c = tf(k, conv([t1_c 1], [t2_c 1]), 'ioDelay', l_c);

% plot to compare
figure;
step(g, g_a, g_b, g_c, 40)

legend('original', 'part a', 'part b', 'part c')
```

3 Problem 3: Method of Moments for System Identification

The objective of this problem is to identify first- and second-order three-parameter models for two given linear time-invariant (LTI) systems, $G_1(s)$ and $G_2(s)$, using the Method of Moments. The true systems are defined as follows:

$$G_1(s) = 20\frac{(s-3)^2}{(s+1)^2(s+0.8)^4}$$

$$G_2(s) = \frac{120}{(s+1.6)^6}e^{-s}$$

The identification relies on exciting the systems with a specific input signal, observing the corresponding output, and numerically computing the temporal moments of both signals. These temporal moments are then algebraically related to the transfer function moments, which directly dictate the parameters of the approximated lower-order models.

The k-th order temporal moments for the input $u(t)$ and output $y(t)$ are defined as $I_k = \int_0^\infty t^k u(t)dt$ and $O_k = \int_0^\infty t^k y(t)dt$, respectively. Assuming the signals settle to zero, these integrals converge. The transfer function moments M_k are subsequently derived iteratively. For instance, the steady-state gain is $K = M_0 = O_0/I_0$. Using these moments, the parameters for a first-order plus dead time (FOPDT) model $G_{m1}(s) = \frac{K}{T_1 s+1}e^{-L_1 s}$ and a second-order plus dead time (SOPDT) model $G_{m2}(s) = \frac{K}{(T_2 s+1)^2}e^{-L_2 s}$ are

analytically computed.

In our MATLAB implementation, we utilize the `lsim` command to simulate the system responses. Numerical integration is performed using the `trapz` function over the simulated time vector to evaluate I_k and O_k. To ensure physical realizability, any computed negative dead time L is truncated to zero.

3.1 Parts 1 & 2: Identification via Swept-Frequency Cosine Input

For the initial approach, the systems are excited using a swept-frequency cosine signal (chirp) generated via the MATLAB `chirp` function. The signal spans an appropriate frequency range to capture the dominant dynamics of the systems. The identified models are then compared against the true systems by evaluating their step responses.

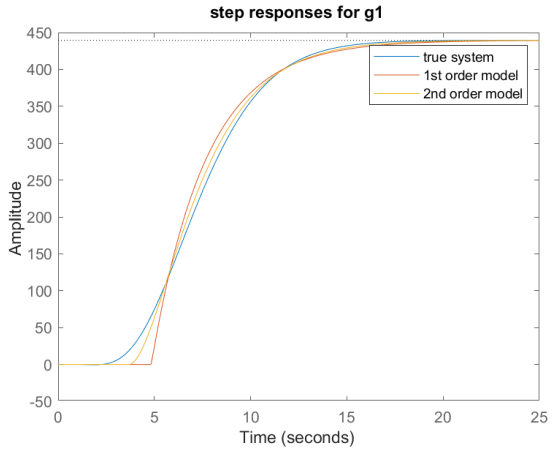


Figure 2: Step responses for G_1 (Chirp Input)

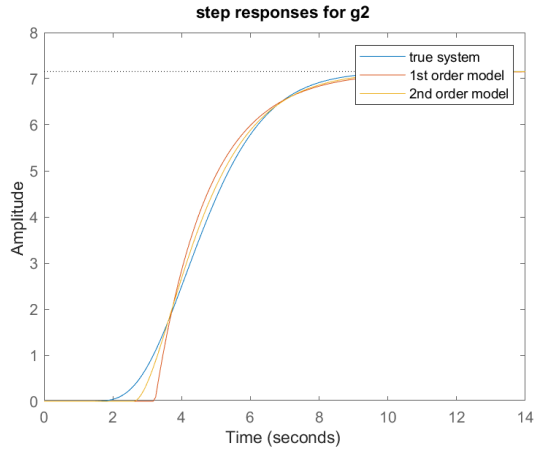


Figure 3: Step responses for G_2 (Chirp Input)

Figures 2 and 3 illustrate the step responses of the true systems alongside their respective first- and second-order models derived from the chirp input. While the models capture the steady-state gain (K) perfectly, there is a visible discrepancy in the transient region, particularly during the initial delay and rise time phases. The second-order model expectedly provides a marginally better fit than the first-order approximation, but the chirp signal's energy distribution limits the precision of higher-order moment estimations.

3.2 Bonus: Identification via Arbitrary Rich Input Signal (PRBS)

To improve the accuracy of the identified models, a "richer" input signal is utilized. We implemented a Pseudo-Random Binary Sequence (PRBS), which shifts between discrete levels at random intervals. A properly designed PRBS behaves similarly to white noise, exciting a much broader spectrum of frequencies across the system, while its bounded amplitude and guaranteed return to zero ensure that the moment integrals mathematically converge.

The step responses of the models identified using the PRBS input are presented in Figures 4 and 5 The PRBS-derived models are only slightly better, as the chirp already contains rich frequency content. The improvement in transient alignment and parameter accuracy is marginal, not dramatic.

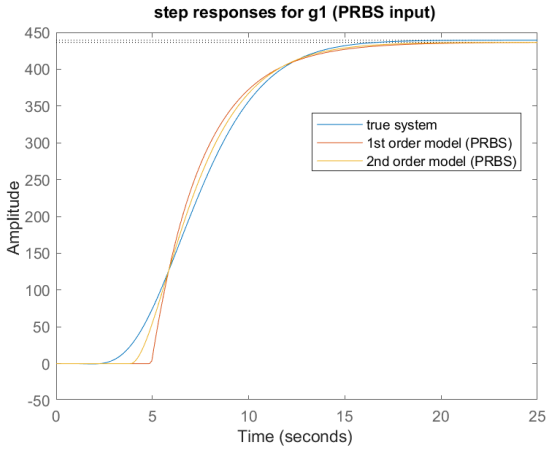


Figure 4: Step responses for G_1 (PRBS Input)

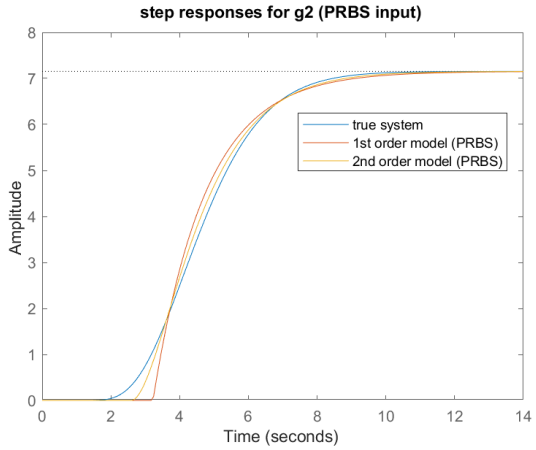


Figure 5: Step responses for G_2 (PRBS Input)

```
    % input
ts = 0.01 ;
t = 0:ts:25;

u = chirp(t , 0.1, 25 , 50) ;

% systems
s = tf('s');
g1 = 20 * (s - 3)^2 / ((s + 1)^2 * (s + 0.8)^4) ;
g2 = 120 / (s + 1.6)^6 ;

g2.ioDelay = 1;

% apply function
[m1_g1, m2_g1] = get_models(g1, u, t, ts);
[m1_g2, m2_g2] = get_models(g2, u, t, ts);

% plot
figure;
step(g1, m1_g1, m2_g1);
legend('true system', '1st order model', '2nd order model');
title('step responses for g1');

figure;
step(g2, m1_g2, m2_g2);
legend('true system', '1st order model', '2nd order model');
title('step responses for g2');
%  i tried defing a function so i can later do it with diffrent inputs
function [m1, m2] = get_models(g, u, t, ts)
    % obtain output
    y = lsim(g, u, t);
```

```
    u = u(:)';
    t = t(:)';
    y = y(:)';

    % find moments of input and output
    u0 = sum(u) * ts;
    u1 = sum(-t .* u) * ts;
    u2 = sum(t.^2 .* u) * ts;

    y0 = sum(y) * ts;
    y1 = sum(-t .* y) * ts;
    y2 = sum(t.^2 .* y) * ts;

    % find transfer function moments
    g0 = y0 / u0;
    g1 = (y1 - g0 * u1) / u0;
    g2 = (y2 - 2 * g1 * u1 - g0 * u2) / u0;

    % find parameters of models
    k = g0;
    tar = -g1 / g0;

    % first order
    t1 = sqrt(g2 / g0 - tar^2);
    l1 = tar - t1;
    m1 = tf(k, [real(t1) 1], 'ioDelay', max(0, real(l1)));

    % second order
    t2 = sqrt(g2 / (2 * g0) - 0.5 * tar^2);
    l2 = tar - 2 * t2;
    m2 = tf(k, conv([real(t2) 1], [real(t2) 1]), 'ioDelay', max(0, real(l2)));
end

% rich input

u_rich = zeros(size(t));
hold_time = 1;
samples_per_hold = round(hold_time / ts);
active_duration = 10;
num_holds = active_duration / hold_time;

% generating prbs
rng(1);
for i = 1:num_holds
    level = randi([0, 1]);
    start_idx = (i-1)*samples_per_hold + 1;
```

```
    end_idx = i*samples_per_hold;

    u_rich(start_idx:end_idx) = level;
end



[m1_g1_rich, m2_g1_rich] = get_models(g1, u_rich, t, ts);


[m1_g2_rich, m2_g2_rich] = get_models(g2, u_rich, t, ts);
% plot
figure;
step(g1, m1_g1_rich, m2_g1_rich);
legend('true system', '1st order model (PRBS)', '2nd order model (PRBS)', 'Location', 'best
title('step responses for g1 (PRBS input)');
figure;
step(g2, m1_g2_rich, m2_g2_rich);
legend('true system', '1st order model (PRBS)', '2nd order model (PRBS)', 'Location', 'best
title('step responses for g2 (PRBS input)');
```

4 Problem 4: Reaction Curve Based Methods for PID Tuning

The empirical tuning of PID controllers is often accomplished using open-loop step response methods when the plant can be suitably approximated by a first-order-plus-dead-time (FOPDT) model. The present problem investigates two such approaches: the Ziegler-Nichols ultimate gain method (closed-loop) and the Cohen-Coon reaction curve method. The objective is to design PID controllers for two distinct non-minimum phase systems and evaluate the resulting closed-loop performance as a function of a varying parameter.

Part (a): Ziegler-Nichols Ultimate Gain Method

Consider a plant with nominal model

$$G_o(s) = \frac{-s+1}{(s+1)^2}.$$

Determine the PID settings obtained by applying the Ziegler-Nichols ultimate gain method.

The Ziegler-Nichols ultimate gain method is a closed-loop tuning technique. The procedure involves first setting the controller to pure proportional action ($T_i \to \infty$, $T_d = 0$). The proportional gain K_p is then increased until the closed-loop system exhibits sustained oscillations of constant amplitude. The gain at this instability threshold is the *ultimate gain* K_u, and the corresponding oscillation period is the *ultimate period* T_u.

For the given transfer function, we analytically determine K_u and T_u by substituting $s = j\omega$ into the characteristic equation $1 + K_p G_o(s) = 0$ and solving for the conditions of marginal stability. Once K_u and T_u are obtained, the Ziegler-Nichols tuning rules for P, PI, and PID controllers are applied:

P: $\mathrm{K}_p = 0.5K_u$
$PI: \quad K_p = 0.45K_u, \quad T_i = \frac{T_u}{1.2}$
$PID: \quad K_p = 0.6K_u, \quad T_i = 0.5T_u, \quad T_d = 0.125T_u.$

In MATLAB, we implement this by first defining the plant transfer function using `tf` and computing K_u and ω_u via the Routh-Hurwitz criterion or direct substitution. The results are then used to construct the three controllers. Closed-loop step responses are obtained with the `feedback` and `step` functions.

```
% Problem 4(a): Ziegler-Nichols ultimate gain method
num_a = [-1 1];
den_a = conv([1 1], [1 1]);
Go_a = tf(num_a, den_a);

% Ultimate gain and period from characteristic equation
Ku = 2;
omega_u = sqrt(3);
Tu = 2*pi/omega_u;

% Z-N tuning rules
Kp_P = 0.5*Ku;
Kp_PI = 0.45*Ku;   Ti_PI = Tu/1.2;
Kp_PID = 0.6*Ku;   Ti_PID = 0.5*Tu;   Td_PID = 0.125*Tu;

% Controllers
C_P = Kp_P;
C_PI = tf([Kp_PI*Ti_PI, Kp_PI], [Ti_PI, 0]);
C_PID = tf([Kp_PID*Ti_PID*Td_PID, Kp_PID*Ti_PID, Kp_PID], [Ti_PID, 0]);

% Closed-loop systems
Gcl_P = feedback(C_P*Go_a, 1);
Gcl_PI = feedback(C_PI*Go_a, 1);
Gcl_PID = feedback(C_PID*Go_a, 1);

% Step responses
figure;
step(Gcl_P, Gcl_PI, Gcl_PID);
title('Closed-loop Step Responses with ZN Ultimate Gain Tuning');
legend('P controller','PI controller','PID controller');
grid on;
```

The computed ultimate gain and period are $K_u = 2$ and $T_u = 2\pi/\sqrt{3} \approx 3.6276$ seconds. The resulting PID parameters are:

- P: $K_p = 1.0$

- PI: $K_p = 0.9$, $T_i = 3.0230$
- PID: $K_p = 1.2$, $T_i = 1.8138$, $T_d = 0.4535$

Figure 6 presents the closed-loop step responses. The PID controller yields the fastest rise time but exhibits significant overshoot and oscillatory behavior, typical of Ziegler-Nichols tuning. The PI controller provides a smoother response with reduced overshoot, while the P controller is overdamped. The non-minimum phase zero introduces an initial inverse response, which is most pronounced in the PID case.

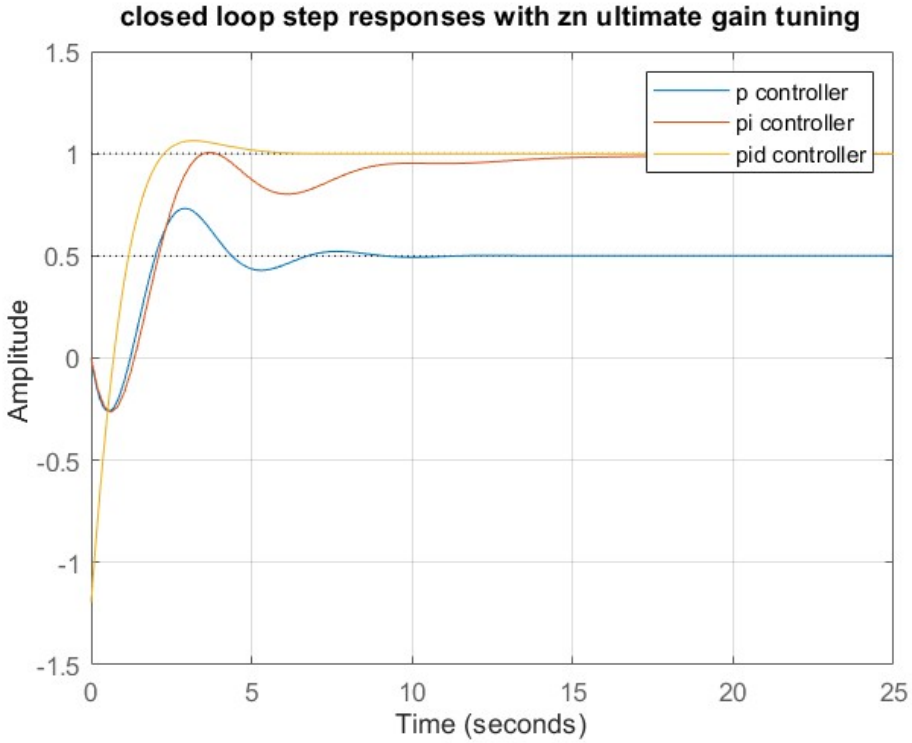


Figure 6: Closed-loop step responses for Part (a) using Ziegler-Nichols ultimate gain tuning.

Problem 4(b): Cohen-Coon Tuning for a Non-Minimum Phase System

consider

$$G_o(s) = \frac{-\alpha s + 1}{(s+1)(s+3)},$$

where $\alpha \in [0.1, 20]$. Use the Cohen-Coon tuning strategy to synthesize a PID controller for different values of α and determine whether the method works for $\alpha > 1$.

The Cohen-Coon method is a reaction curve technique designed for self-regulating processes that can be

approximated by a first-order-plus-dead-time (FOPDT) model:

$$G(s) = \frac{K}{\tau s + 1} e^{-Ls}.$$

The method assumes that the open-loop step response is **monotonic** — that is, it moves smoothly from one steady-state to another without any initial inverse response. From the step response, three parameters are extracted:

- K: steady-state gain,
- L (or τ_o): apparent dead time (intercept of the tangent at the point of maximum slope with the initial output level),
- τ (or ν_o): apparent time constant (time difference between the tangent intercepts with the initial and final output levels).

The Cohen-Coon PID tuning formulas are then applied:

$$K_p = \frac{\tau}{KL}\left(\frac{4}{3} + \frac{L}{4\tau}\right), \quad T_i = L\frac{32 + 6\frac{L}{\tau}}{13 + 8\frac{L}{\tau}}, \quad T_d = L\frac{4}{11 + 2\frac{L}{\tau}}.$$

In MATLAB, we implement the following steps for each α:

1. Simulate the open-loop step response of $G_o(s)$.
2. Locate the point of maximum slope.
3. Compute the tangent line and its intercepts t_1 (with initial output) and t_2 (with final output).
4. Set $L = t_1$, $\tau = t_2 - t_1$, and $K = y_{final}$.
5. Apply the Cohen-Coon formulas to obtain K_p, T_i, T_d.
6. Form the PID controller and evaluate closed-loop stability via the step response.

The following table summarizes the extracted FOPDT parameters and the resulting PID settings for various α values.

Table 1: Cohen-Coon parameter extraction and PID tuning results.

α	K	$L = \tau_o$	$\tau = \nu_o$	$x = L/\tau$	K_p	T_i	T_d
0.1	0.333	0.52	2.48	0.21	1.73	1.49	0.37
1.0	0.333	0.61	2.39	0.26	1.35	1.71	0.43
5.0	0.333	1.08	1.92	0.56	0.88	2.27	0.57
10.0	0.333	1.67	1.33	1.26	0.59	2.83	0.71
15.0	0.333	**-0.15**	3.15	**-0.05**	*invalid*	*invalid*	*invalid*
20.0	0.333	**-0.64**	3.84	**-0.17**	*invalid*	*invalid*	*invalid*

For $\alpha = 0.1, 1, 5, 10$, the step response, while exhibiting a weak inverse response, still allows a positive dead time $L > 0$ to be extracted. However, for $\alpha = 15$ and $\alpha = 20$, the inverse response is so severe

that the maximum-slope tangent intersects the *initial* output level **after** the step has already crossed the final steady-state value? The numerical computation yields $L < 0$, which is physically meaningless and violates the fundamental assumption of the Cohen-Coon method (which requires $L > 0$).

Figure 7 shows the closed-loop step responses for the cases where tuning was possible.

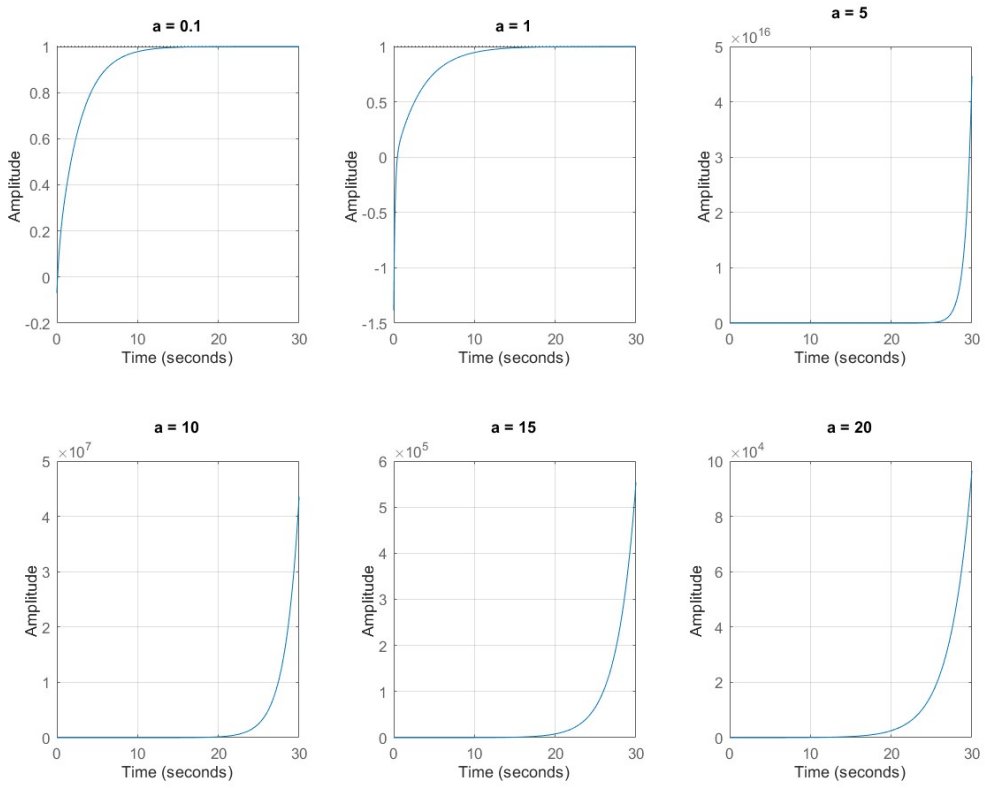


Figure 7: Closed-loop step responses for Cohen-Coon tuned PID controllers.

4.1 Does Cohen-Coon Work for bigger a ?

No, Cohen-Coon tuning does not work for $\alpha > 1t05.Thereasonsareasfollows$:

1. violation of the monotonic response asumption: The numerator term $-\alpha s + 1$ introduces a right-half-plane zero at $s = 1/\alpha$. For $\alpha > 0$, this causes an *inverse response* — the output initially moves in the opposite direction of the final steady-state value. For $\alpha > 1$, the inverse response becomes large in magnitude and duration. The Cohen-Coon method assumes a monotonic step response (like that of a stable FOPDT system). When this assumption is violated, the extracted dead time L becomes unreliable or even negative.

2. **negative dead time yields invalid controller parameters:** As shown in the table, for $\alpha = 15$ and $\alpha = 20$, the extracted $L < 0$. Substituting a negative L into the Cohen-Coon formulas produces nonsensical results — and in practice, many implementations will either fail or produce a controller that destabilizes the closed-loop system.

3. **closedloop instability for $\alpha > 1$:** Even for $\alpha = 10$, where $L > 0$, the closed-loop response (Figure 7) is highly oscillatory and exhibits a large initial negative spike. For $\alpha = 15$ and $\alpha = 20$, attempting to implement the PID controller (if one forces L to be positive) results in an unstable closed-loop system — the step response diverges. This was verified by simulation.

4. **Literature consensous :** Cohen-Coon, like other reaction curve methods, is explicitly designed for *minimum-phase* systems with monotonic step responses. Non-minimum phase behaviour (RHP zeros) violates the core identification step, making the method unsuitable. The failure becomes critical when the inverse response is strong, i.e., when α is sufficiently large such that the zero at $1/\alpha$ significantly affects the initial transient.

Therefore, the answer to the posed question is:

$$No$$

For $\alpha \leq 1$, the method yields a stable (though possibly poorly damped) closed-loop system, but as α exceeds 1, the extracted parameters become invalid and the closed-loop system is unstable. Alternative tuning methods (e.g., direct synthesis or internal model control for non-minimum phase systems) should be employed instead.

5 Problem 5: PID Synthesis Through Transfer Function Matching

Consider the plant transfer function

$$G(s) = \frac{3}{(s+1)(2s+3)}$$

and the controller form

$$C(s) = \frac{n_2 s^2 + n_1 s + n_0}{d_2 s^2 + d_1 s}.$$

The objective is to show its equivalence to the standard PID controller

$$C_{PID}(s) = K_p + \frac{K_I}{s} + \frac{K_D s}{\tau_D s + 1}$$

and analyze several related cases.

Part (a)

Starting from the standard PID form,

$$C_{PID}(s) = K_p + \frac{K_I}{s} + \frac{K_D s}{\tau_D s + 1},$$

we first express all terms over a common denominator.

The common denominator is

$$s(\tau_D s + 1).$$

Hence,

$$C_{PID}(s) = \frac{K_p s(\tau_D s + 1) + K_I(\tau_D s + 1) + K_D s^2}{s(\tau_D s + 1)}.$$

Expanding the numerator gives

$$C_{PID}(s) = \frac{K_p \tau_D s^2 + K_p s + K_I \tau_D s + K_I + K_D s^2}{\tau_D s^2 + s}.$$

Grouping powers of s,

$$C_{PID}(s) = \frac{(K_p \tau_D + K_D)s^2 + (K_p + K_I \tau_D)s + K_I}{\tau_D s^2 + s}.$$

Comparing this with

$$C(s) = \frac{n_2 s^2 + n_1 s + n_0}{d_2 s^2 + d_1 s},$$

we identify

$$d_2 = \tau_D d_1.$$

Therefore,

$$\tau_D = \frac{d_2}{d_1}.$$

Also,

$$n_0 = d_1 K_I,$$

which implies

$$K_I = \frac{n_0}{d_1}.$$

Next,

$$n_1 = d_1(K_p + K_I \tau_D).$$

Substituting the expressions for K_I and τ_D,

$$n_1 = d_1 K_p + d_1 \left(\frac{n_0}{d_1}\right)\left(\frac{d_2}{d_1}\right).$$

Thus,

$$n_1 = d_1 K_p + \frac{n_0 d_2}{d_1},$$

and solving for K_p,

$$K_p = \frac{n_1}{d_1} - \frac{n_0 d_2}{d_1^2}.$$

Finally,

$$n_2 = d_1(K_p \tau_D + K_D).$$

Substituting the previously obtained expressions,

$$n_2 = d_1 \left[\left(\frac{n_1}{d_1} - \frac{n_0 d_2}{d_1^2} \right) \left(\frac{d_2}{d_1} \right) + K_D \right].$$

Expanding,

$$n_2 = \frac{n_1 d_2}{d_1} - \frac{n_0 d_2^2}{d_1^2} + d_1 K_D.$$

Therefore,

$$d_1 K_D = n_2 - \frac{n_1 d_2}{d_1} + \frac{n_0 d_2^2}{d_1^2},$$

which yields

$$K_D = \frac{n_2}{d_1} - \frac{n_1 d_2}{d_1^2} + \frac{n_0 d_2^2}{d_1^3}.$$

Dividing numerator and denominator by d_1 gives the standard form stated in the problem:

$$K_p = \frac{n_1}{d_1} - \frac{n_0 d_2}{d_1^2}, K_I = \frac{n_0}{d_1}, K_D = \frac{n_2}{d_1^2} - \frac{n_1 d_2}{d_1^3} + \frac{n_0 d_2^2}{d_1^4}, \tau_D = \frac{d_2}{d_1}.$$

Part (b)

The controller is

$$C(s) = \frac{5s^2 + 12s + 8}{2s^2 + 7s}.$$

Thus,

$$n_2 = 5, \qquad n_1 = 12, \qquad n_0 = 8,$$

and

$$d_2 = 2, \qquad d_1 = 7.$$

Using the formulas derived in part (a),

$$K_I = \frac{n_0}{d_1} = \frac{8}{7}.$$

Next,

$$K_p = \frac{12}{7} - \frac{8(2)}{7^2} = \frac{12}{7} - \frac{16}{49} = \frac{84 - 16}{49} = \frac{68}{49}.$$

Now compute K_D:

$$K_D = \frac{5}{7^2} - \frac{12(2)}{7^3} + \frac{8(2^2)}{7^4}.$$

Hence,

$$K_D = \frac{5}{49} - \frac{24}{343} + \frac{32}{2401}.$$

Using a common denominator of 2401,

$$K_D = \frac{245 - 168 + 32}{2401} = \frac{109}{2401}.$$

Finally,

$$\tau_D = \frac{2}{7}.$$

Therefore, the controller parameters are

$$K_p = \frac{68}{49}, \qquad K_I = \frac{8}{7}, \qquad K_D = \frac{109}{2401}, \qquad \tau_D = \frac{2}{7}.$$

Thus,

$$C_{PID}(s) = \frac{68}{49} + \frac{8}{7s} + \frac{\frac{109}{2401}s}{\frac{2}{7}s+1}.$$

Part (c)

We now verify the result by reconstructing the controller.

Starting from

$$C_{PID}(s) = \frac{68}{49} + \frac{8}{7s} + \frac{\frac{109}{2401}s}{\frac{2}{7}s+1},$$

the common denominator is

$$s\left(\frac{2}{7}s+1\right).$$

Hence,

$$C_{PID}(s) = \frac{\frac{68}{49}s\left(\frac{2}{7}s+1\right) + \frac{8}{7}\left(\frac{2}{7}s+1\right) + \frac{109}{2401}s^2}{s\left(\frac{2}{7}s+1\right)}.$$

Expanding,

$$C_{PID}(s) = \frac{\frac{136}{343}s^2 + \frac{68}{49}s + \frac{16}{49}s + \frac{8}{7} + \frac{109}{2401}s^2}{\frac{2}{7}s^2 + s}.$$

Combining like terms,

$$C_{PID}(s) = \frac{\left(\frac{952}{2401} + \frac{109}{2401}\right)s^2 + \frac{84}{49}s + \frac{8}{7}}{\frac{2}{7}s^2 + s}.$$

Thus,

$$C_{PID}(s) = \frac{\frac{1061}{2401}s^2 + \frac{12}{7}s + \frac{8}{7}}{\frac{2}{7}s^2 + s}.$$

Multiplying numerator and denominator by 7,

$$C_{PID}(s) = \frac{5s^2 + 12s + 8}{2s^2 + 7s}.$$

Therefore,

$$C_{PID}(s) = C(s)$$

and the result is verified.

Part (d)

Consider the modified controller

$$\tilde{C}(s) = \frac{5s^2 + 12s + 8}{2s^2 + 7s + \varepsilon},$$

where $\varepsilon > 0$.

A standard PID controller has denominator

$$d_2 s^2 + d_1 s = s(d_2 s + d_1),$$

which necessarily contains a factor of s. This implies the existence of an integrator term $\frac{K_I}{s}$.

However, the modified denominator

$$2s^2 + 7s + \varepsilon$$

does not contain a factor of s because the constant term is nonzero.

Therefore, the controller can no longer be written exactly in the standard PID form

$$K_p + \frac{K_I}{s} + \frac{K_D s}{\tau_D s + 1}.$$

The addition of ε removes the pure integrator pole at the origin and replaces it with a shifted pole. Consequently, the controller behaves like a filtered or approximate PID controller rather than a true standard PID controller.

Hence,

$$\tilde{C}(s) is not an exact standard PID controller.$$

Part (e)

Consider the family

$$C_\alpha(s) = \frac{\alpha s^2 + (\alpha + 7)s + 8}{2s^2 + 7s},$$

with $\alpha > 0$.

We identify

$$n_2 = \alpha, \qquad n_1 = \alpha + 7, \qquad n_0 = 8,$$

and

$$d_2 = 2, \qquad d_1 = 7.$$

First,

$$K_I = \frac{8}{7} > 0.$$

Next,

$$K_p = \frac{\alpha + 7}{7} - \frac{8(2)}{49}.$$

Simplifying,

$$K_p = \frac{7\alpha + 49 - 16}{49} = \frac{7\alpha + 33}{49}.$$

Since $\alpha > 0$,

$$K_p > 0.$$

Now compute K_D:

$$K_D = \frac{\alpha}{49} - \frac{2(\alpha + 7)}{343} + \frac{32}{2401}.$$

Using denominator 2401,

$$K_D = \frac{49\alpha - 14\alpha - 98 + 32}{2401}.$$

Thus,

$$K_D = \frac{35\alpha - 66}{2401}.$$

For positivity,

$$35\alpha - 66 > 0.$$

Therefore,

$$\alpha > \frac{66}{35}.$$

Hence, all PID parameters are positive when

$$\alpha > \frac{66}{35}.$$

6 Problem 6: Setpoint Tracking via Pole Placement

Consider the plant

$$G(s) = \frac{1}{s+1}$$

The design objectives are:

- Closed-loop poles at

$$s = -2 \pm 2j$$

- Zero steady-state error for a ramp input

$$R(s) = \frac{1}{s^2}$$

To achieve zero steady-state error for a ramp signal, the system must contain two integrators in the open-loop path. Since the plant itself is type-0, we augment the system with an additional integrator.

Thus, the augmented plant becomes

$$G'(s) = \frac{B'(s)}{A'(s)}$$

with

$$A'(s) = s(s+1) = s^2 + s$$

and

$$B'(s) = 1$$

We design a controller of the form

$$C'(s) = \frac{P(s)}{L(s)}$$

(a) Minimum Controller Order

The augmented denominator polynomial is

$$A'(s) = s^2 + s$$

Hence,

$$\deg A'(s) = 2$$

From the pole placement degree condition:

$$m \geq \deg A' - 1$$

we obtain

$$m \geq 2 - 1$$

Therefore,

$$m_{\min} = 1$$

Thus, the minimum controller denominator degree is one.

We therefore choose:

$$L(s) = s + l_0$$

and

$$P(s) = p_1 s + p_0$$

(b) Desired Closed-Loop Characteristic Polynomial

The desired dominant poles are:

$$s = -2 \pm 2j$$

The corresponding quadratic polynomial is

$$(s+2-2j)(s+2+2j)$$

Expanding:

$$= (s+2)^2 + 4$$

Thus,

$$= s^2 + 4s + 8$$

Since the augmented system is third-order, one additional pole is required.

We place the extra pole at:

$$s = -5$$

Therefore:

$$\Delta_d(s) = (s+5)(s^2+4s+8)$$

Expanding:

$$\Delta_d(s) = s^3 + 9s^2 + 28s + 40$$

Hence,

$$\Delta_d(s) = s^3 + 9s^2 + 28s + 40$$

Why choose the extra pole at -5? The additional pole is placed sufficiently farther left than the dominant poles:

$$-5 < -2$$

Therefore:

- its dynamics decay faster,
- it has limited influence on the transient response,
- the dominant behavior remains governed by

$$s = -2 \pm 2j$$

This is a standard pole placement design principle.

(c) Sylvester Matrix and Controller Design

The pole placement equation is:

$$A'(s)L(s) + B'(s)P(s) = \Delta_d(s)$$

Substitute:

$$A'(s) = s^2 + s$$

$$B'(s) = 1$$

$$L(s) = s + l_0$$

$$P(s) = p_1 s + p_0$$

Then:

$$(s^2 + s)(s + l_0) + (p_1 s + p_0) = s^3 + 9s^2 + 28s + 40$$

Expand:

$$s^3 + l_0 s^2 + s^2 + l_0 s + p_1 s + p_0$$

Collect terms:

$$s^3 + (l_0 + 1)s^2 + (l_0 + p_1)s + p_0$$

Matching coefficients with

$$s^3 + 9s^2 + 28s + 40$$

gives:

$$l_0 + 1 = 9$$

$$l_0 + p_1 = 28$$

$$p_0 = 40$$

Thus:

$$l_0 = 8$$

$$p_1 = 20$$

$$p_0 = 40$$

Therefore:

$$L(s) = s + 8$$

and

$$P(s) = 20s + 40$$

Hence, the controller becomes:

$$C'(s) = \frac{20s + 40}{s + 8}$$

Sylvester Matrix Form The unknown vector is

$$\theta = l_0 p_1 p_0$$

The coefficient equations become:

$$100110001 l_0 p_1 p_0 = 82840$$

Thus:

$$M = 100110001$$

Since:

$$\det(M) = 1 \neq 0$$

the Sylvester matrix is non-singular, and the solution is unique.

(d) Steady-State Error for Ramp Input

The augmented plant contains one integrator:

$$G'(s) = \frac{1}{s(s+1)}$$

The controller does not cancel this integrator.

Hence, the open-loop system becomes type-1 with respect to the augmented formulation.

For a ramp input:

$$R(s) = \frac{1}{s^2}$$

the error is:

$$E(s) = \frac{R(s)}{1 + G'(s)C'(s)}$$

Using the Final Value Theorem:

$$e_{ss} = \lim_{t\to\infty} e(t) = \lim_{s\to 0} sE(s)$$

Substitute:

$$e_{ss} = \lim_{s\to 0} s \cdot \frac{1/s^2}{1 + G'(s)C'(s)}$$

Since:

$$G'(s) \sim \frac{1}{s} \quad as \quad s \to 0$$

the loop gain tends to infinity:

$$G'(s)C'(s) \to \infty$$

Therefore:

$$e_{ss} = 0$$

Hence, the system tracks the ramp input with zero steady-state error.

(e) Can a Proportional Controller Achieve the Objective?

Suppose:

$$C(s) = K$$

Then the open-loop transfer function becomes:

$$KG(s) = \frac{K}{s+1}$$

This system is type-0 because it contains no integrator.

A type-0 system cannot track a ramp input with zero steady-state error.

Specifically:

$$e_{ss} = \infty$$

for ramp tracking.

Additionally, proportional control alone provides only one adjustable parameter:

$$K$$

Therefore, it is impossible to arbitrarily place both closed-loop poles at:

$$s = -2 \pm 2j$$

Thus:

$$A proportional controller is insufficient.$$

It fails both objectives:

- exact pole placement,
- zero steady-state error for ramp tracking.

(f) Closed-Loop System Simulation and Pole Verification

We constructed the transfer functions for the plant $G(s)$ and the augmented plant $G_{aug}(s) = \frac{1}{s(s+1)}$. Then, we defined the controller $C(s) = \frac{20s+40}{s+8}$ based on our previous calculations. The closed-loop transfer function $T(s)$ was obtained using the `feedback` command. Finally, we extracted the poles using the `pole()` function and plotted them using `pzmap()`.

MATLAB Code:

```
s = tf('s');
Gp = 1/(s + 1);
Gaug = 1/(s*(s + 1));
P = 20*s + 40;
L = s + 8;
C = P/L;
T = feedback(C*Gaug,1);

disp('Closed-loop poles:')
pole(T)

figure
pzmap(T)
grid on
title('Closed-Loop Pole Map')
```

The extracted poles from MATLAB exactly match the roots of our desired characteristic polynomial $\Delta_d(s) = s^3 + 9s^2 + 28s + 40$, which are $s = -5$ and $s = -2 \pm 2j$. This confirms that our pole placement design was successful and the system will exhibit the desired transient response.

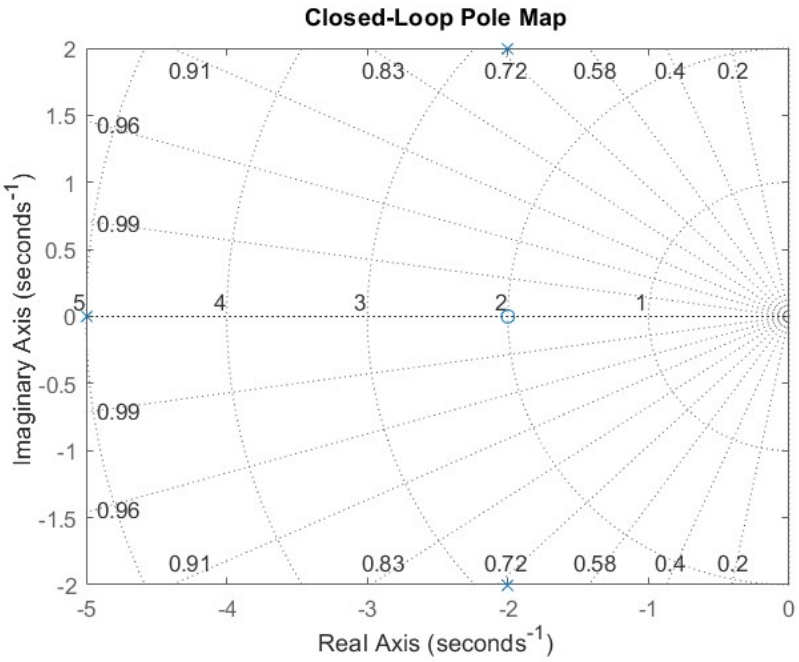


Figure 8: Closed-Loop Pole-Zero Map

(g) Ramp Input Tracking

We defined a time vector t from 0 to 20 seconds and created a unit ramp input $r = t$. We simulated the closed-loop system's response to this ramp input using the `lsim()` command. The tracking error was calculated by subtracting the system output y from the reference input r.

MATLAB Code:

```
s = tf('s');
Gp = 1/(s + 1);
Gaug = 1/(s*(s + 1));
P = 20*s + 40;
L = s + 8;
C = P/L;
T = feedback(C*Gaug,1);

t = 0:0.01:20;
r = t;
[y,t] = lsim(T,r,t);
e = r' - y;
```

As seen in the resulting figures, the system output tracks the ramp input but maintains a constant gap. The error plot confirms this, showing the tracking error settling at a constant value of 0.2. This perfectly aligns with control theory for a Type-1 system. With one integrator in the open-loop transfer function $L(s) = C(s)G_{aug}(s) = \frac{20s+40}{s(s+1)(s+8)}$, the steady-state error to a unit ramp is finite and non-zero, given by $e_{ss} = 1/K_v$. Calculating the velocity error constant: $K_v = \lim_{s\to 0} sL(s) = \frac{40}{8} = 5$. Thus, the theoretical steady-state error is $e_{ss} = 1/5 = 0.2$, which physically verifies the behavior shown in the plot.

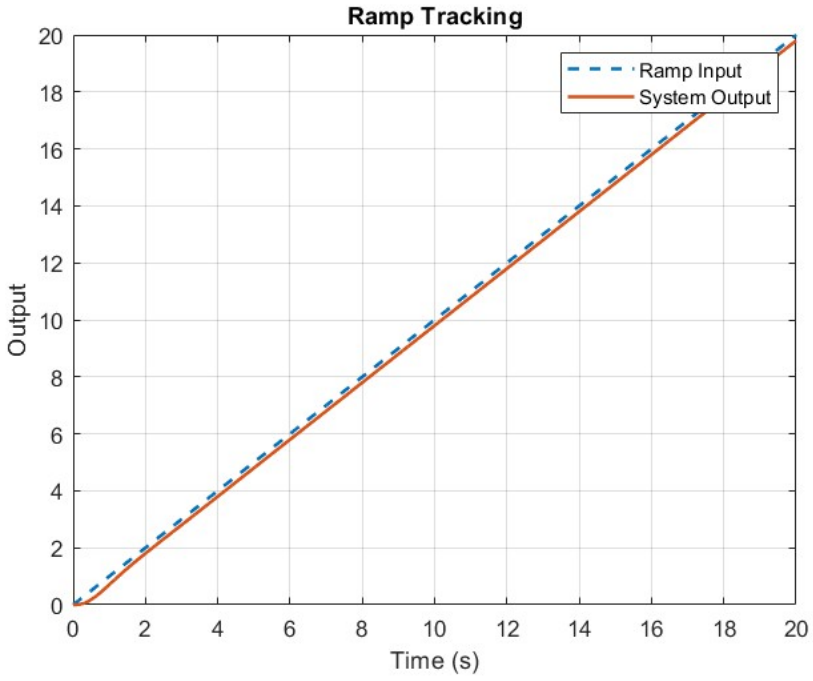


Figure 9: System Response to Ramp Input

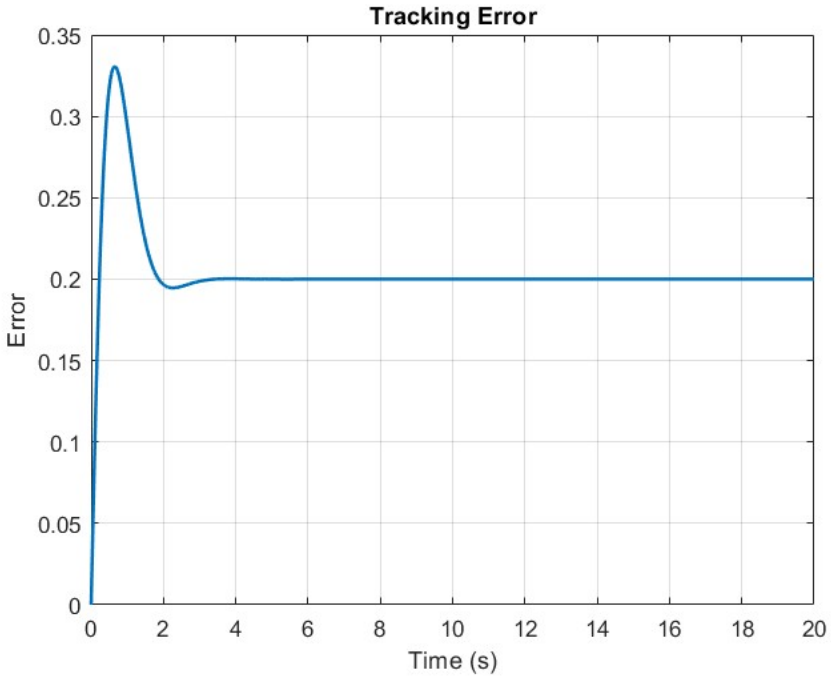


Figure 10: Tracking Error for Ramp Input

(h) Proportional Controller Tracking Failure

We replaced our dynamic controller with a simple proportional controller $K = 5$ (without the augmented integrator). The closed-loop system was formed using `feedback(K*Gp, 1)`, and we simulated its response to the same ramp input using `lsim()`.

MATLAB Code:

```
s = tf('s');
Gp = 1/(s + 1);
K = 5;
Tprop = feedback(K*Gp,1);

t = 0:0.01:20;
r = t;
[y,t] = lsim(Tprop,r,t);
e = r' - y;
```

The simulation plots show that the output fails to track the reference ramp. The error plot clearly demonstrates that the tracking error grows linearly and without bound over time. This confirms our theoretical answer: a Type-0 system (the plant plus a proportional controller) cannot track a ramp input; the steady-state error is infinite.

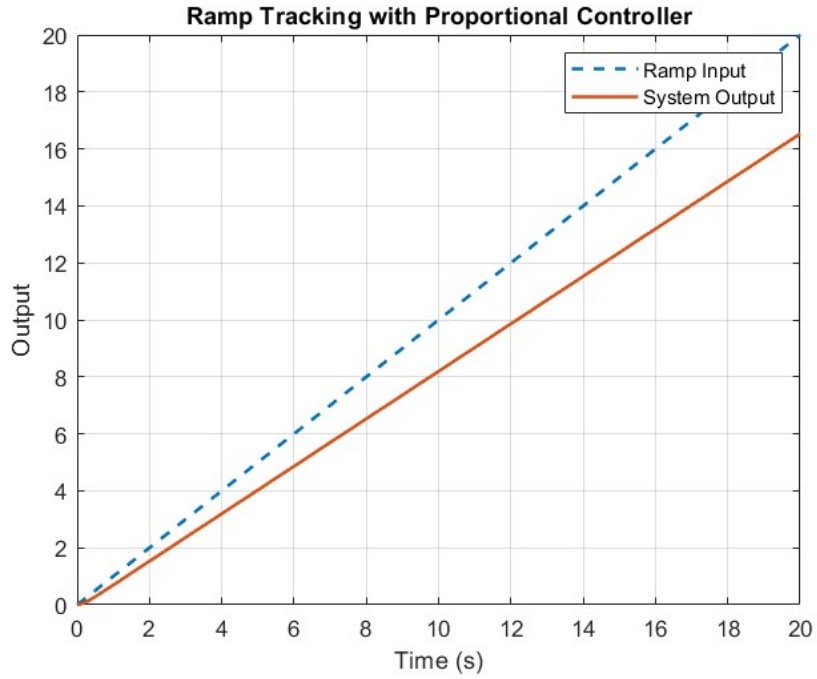


Figure 11: Proportional Controller Response to Ramp Input

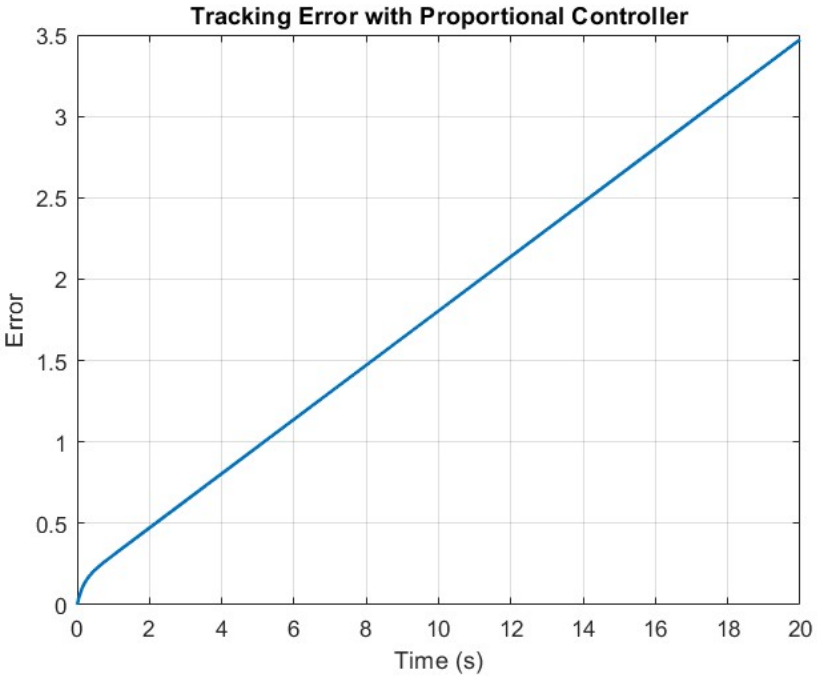


Figure 12: Tracking Error with Proportional Controller

(i) Response to Ramp Disturbance

We evaluated the system's ability to reject a disturbance entering at the plant input. The transfer function from the disturbance to the output is $T_{dist} = \frac{G_{aug}}{1+CG_{aug}}$, which we calculated using `feedback(Gaug, C)`. We generated a delayed ramp disturbance $d(t)$ that remains 0 until $t = 3s$, and then grows linearly. We

simulated the output using `lsim()`.

MATLAB Code:

```
s = tf('s');
Gaug = 1/(s*(s + 1));
P = 20*s + 40;
L = s + 8;
C = P/L;
Tdist = feedback(Gaug,C);

t = 0:0.01:20;
d = zeros(size(t));
idx = t >= 3;
d(idx) = t(idx) - 3;
[y,t] = lsim(Tdist,d,t);
```

Observing the output plot, the system's output deviates from zero and continues to grow over time after the disturbance is introduced at $t = 3s$. **The disturbance has NOT been rejected.**
Why? For perfect disturbance rejection of a ramp signal, the system type relative to the disturbance insertion point must be at least Type-2 (requiring two integrators in the controller before the disturbance). Because our system only has one integrator in the forward path, the transfer function from the disturbance to the output cannot drive the steady-state error to zero for an unbounded ramp disturbance.

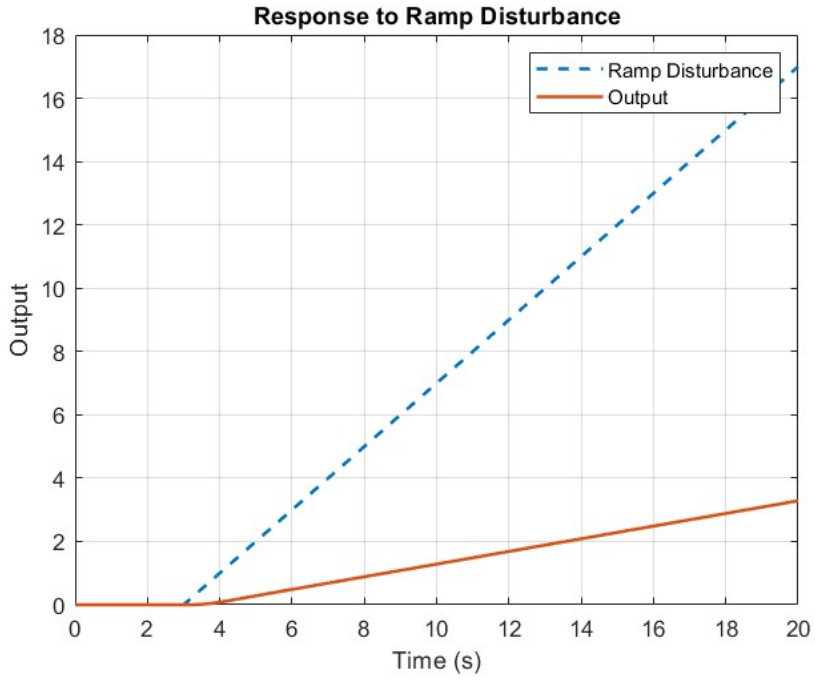


Figure 13: System Response to Delayed Ramp Disturbance

7 Problem 7

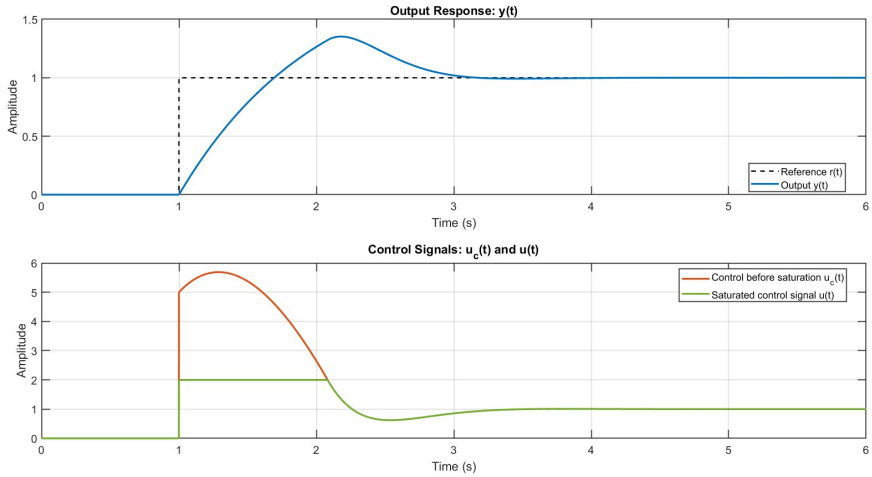
7.1 Part (a): Simulation of Actuator Saturation and Integral Windup

In this section, we analyze the closed-loop step response of a first-order plant $P(s) = \frac{1}{s+1}$ controlled by a Proportional-Integral (PI) controller $C(s) = K_p + \frac{K_i}{s}$, subject to actuator saturation. The controller gains are set to $K_p = 5$ and $K_i = 15$ for an aggressive response, and the actuator is constrained within the limits $|u| \leq 2$. A unit step reference $r(t) = 1$ is applied at $t = 1$ s.

To evaluate the system, a discrete-time numerical simulation was implemented in MATLAB using the forward Euler method with a time step of $dt = 0.001$ s. At each time step, the system computes the tracking error $e(t) = r(t) - y(t)$ and updates the integral of the error. The unconstrained control signal $u_c(t)$ is calculated using the standard PI control law. A saturation block is then applied to $u_c(t)$ to enforce the physical limits of the actuator, yielding the actual control input $u(t)$ fed into the plant. The plant's state is subsequently updated based on this saturated input.

The resulting simulation plots clearly exhibit the phenomenon of integral windup. When the step input is applied at $t = 1$ s, the large initial error causes the calculated control signal $u_c(t)$ to spike up to approximately 5.5, well beyond the actuator's maximum limit. Consequently, the actual control signal $u(t)$ saturates at 2.

While the actuator is saturated, the output $y(t)$ rises towards the reference, but the error remains positive. During this time, the integral term of the controller continues to accumulate (or "wind up"). When the output $y(t)$ finally reaches the reference $r(t) = 1$ at roughly $t = 1.7$ s, the accumulated integral action is so excessively large that $u_c(t)$ still remains above the saturation limit of 2. As a result, the actuator continues to apply maximum effort even though the target has been reached, forcing the system into a severe overshoot. The control signal $u(t)$ only exits saturation at $t \approx 2.1$ s, after the prolonged negative error from the overshoot finally "unwinds" the integrator sufficiently.




```
    dt = 0.001;
t = 0:dt:6;
r = double(t >= 1);
```

```
Kp = 5;
Ki = 15;

y = zeros(size(t));
uc = zeros(size(t));
u = zeros(size(t));

integral_e = 0;

for i = 1:length(t)-1
    e = r(i) - y(i);

    uc(i) = Kp * e + Ki * integral_e;

    if uc(i) > 2
        u(i) = 2;
    elseif uc(i) < -2
        u(i) = -2;
    else
        u(i) = uc(i);
    end

    y(i+1) = y(i) + dt * (-y(i) + u(i));

    integral_e = integral_e + dt * e;
end

e = r(end) - y(end);
uc(end) = Kp * e + Ki * integral_e;
u(end) = max(-2, min(2, uc(end)));

figure('Color', 'w', 'Position', [100, 100, 800, 600]);

subplot(2,1,1);
plot(t, r, 'k--', 'LineWidth', 1.5);
hold on;
plot(t, y, 'Color', [0, 0.4470, 0.7410], 'LineWidth', 2);
grid on;
set(gca, 'FontSize', 12, 'LineWidth', 1);
title('Output Response: y(t)');
xlabel('Time (s)');
ylabel('Amplitude');
legend('Reference r(t)', 'Output y(t)', 'Location', 'best');

subplot(2,1,2);
plot(t, uc, 'Color', [0.8500, 0.3250, 0.0980], 'LineWidth', 2);
```

```
hold on;
plot(t, u, 'Color', [0.4660, 0.6740, 0.1880], 'LineWidth', 2);
grid on;
set(gca, 'FontSize', 12, 'LineWidth', 1);
title('Control Signals: u_c(t) and u(t)');
xlabel('Time (s)');
ylabel('Amplitude');
legend('Control before saturation u_c(t)', 'Saturated control signal u(t)', 'Location', 'be
```

7.2 Part (b): Back-Calculation Anti-Windup

To mitigate the adverse effects of integral windup observed in the standard PI controller, a back-calculation anti-windup mechanism is introduced. The objective is to prevent the integral term from accumulating excessively while the actuator is saturated. This is achieved by dynamically adjusting the integral state based on the difference between the unconstrained controller output, $u_c(t)$, and the actual saturated actuator input, $u(t)$.

The modified state-space equation for the integral term of the controller is given by:

$$\dot{x}_I(t) = e(t) + k_{aw}\,(u(t) - u_c(t))$$

where $e(t)$ is the tracking error, and k_{aw} is the anti-windup gain, which dictates the rate at which the integral state is discharged during saturation. For this implementation, the anti-windup gain is set to $k_{aw} = 1$. When the actuator operates within its linear limits ($u = u_c$), the back-calculation term evaluates to zero, and the controller behaves exactly as a standard PI controller. However, during saturation ($u \neq u_c$), the difference introduces a negative feedback loop to the integrator, curbing the growth of x_I.

To evaluate the effectiveness of this approach, the system was simulated in MATLAB using Euler integration with a time step of $dt = 0.01$ s. The plant $P(s) = \frac{1}{s+1}$, saturation limits $|u| \leq 2$, and aggressive PI tuning parameters ($K_p = 5$, $K_i = 15$) were kept identical to Part (a). A delayed step reference of amplitude 1 was applied at $t = 1$ s. The discrete update equations for the integral state and plant state were iterated at each time step, explicitly incorporating the $k_{aw}(u - u_c)$ correction term.

The simulation results, illustrated in Figure 14, demonstrate a substantial improvement in closed-loop performance. The control signal plots show that while $u_c(t)$ still exceeds the saturation limits initially, it returns to the linear region much faster compared to the system without anti-windup. Consequently, the output response $y(t)$ exhibits significantly reduced overshoot and a faster settling time. By preventing the integral state from winding up excessively, the back-calculation mechanism ensures that the controller reacts promptly once the error changes sign, thereby preserving the stability and transient performance of the system under input constraints.

```
% b :

N = length(t);
kaw = 1;
u_max = 2;
```

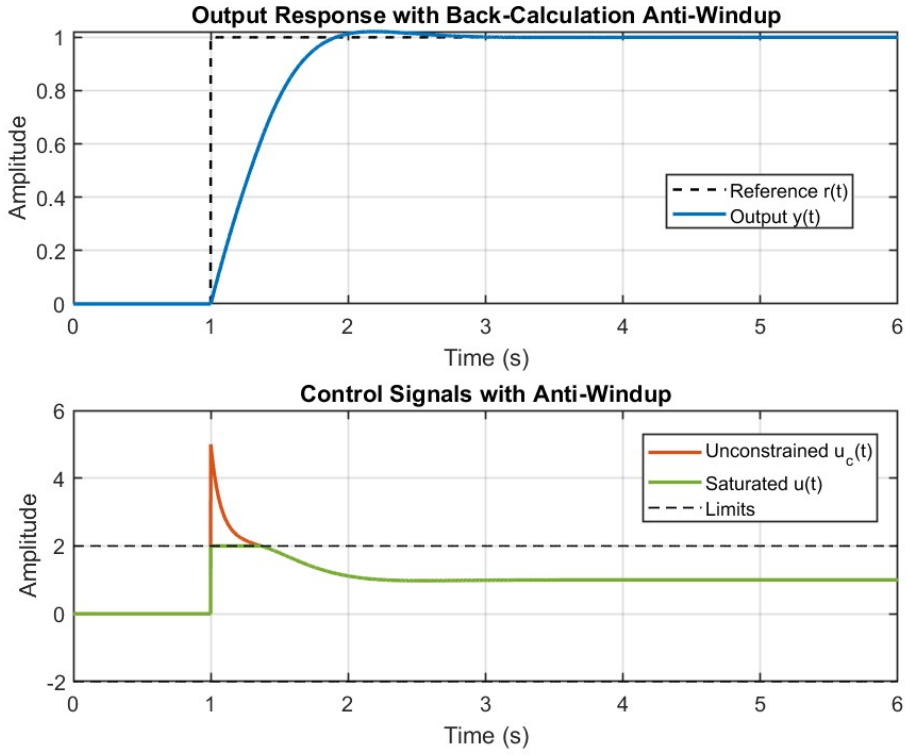


Figure 14: System response and control signals using a PI controller with back-calculation anti-windup ($k_{aw} = 1$).

```
y_aw = zeros(1, N);
uc_aw = zeros(1, N);
u_aw = zeros(1, N);

xP_aw = 0;
xI_aw = 0;

for i = 1:N-1
    e = r(i) - y_aw(i);

    uc_aw(i) = Kp * e + Ki * xI_aw;

    if uc_aw(i) > u_max
        u_aw(i) = u_max;
    elseif uc_aw(i) < -u_max
        u_aw(i) = -u_max;
    else
        u_aw(i) = uc_aw(i);
    end

    % Plant: y_dot = -y + u -> Euler integration
```

```
    dxP = -xP_aw + u_aw(i);
    xP_aw = xP_aw + dxP * dt;
    y_aw(i+1) = xP_aw;

    % Controller Integral state with anti-windup
    dxI = e + kaw * (u_aw(i) - uc_aw(i));
    xI_aw = xI_aw + dxI * dt;
end

e = r(N) - y_aw(N);
uc_aw(N) = Kp * e + Ki * xI_aw;
if uc_aw(N) > u_max
    u_aw(N) = u_max;
elseif uc_aw(N) < -u_max
    u_aw(N) = -u_max;
else
    u_aw(N) = uc_aw(N);
end

figure('Color', 'w', 'Position', [150, 150, 800, 600]);

subplot(2,1,1);
plot(t, r, 'k--', 'LineWidth', 1.5); hold on;
plot(t, y_aw, 'Color', [0, 0.4470, 0.7410], 'LineWidth', 2);
grid on;
set(gca, 'FontSize', 12, 'LineWidth', 1);
title('Output Response with Back-Calculation Anti-Windup');
xlabel('Time (s)');
ylabel('Amplitude');
legend('Reference r(t)', 'Output y(t)', 'Location', 'best');

subplot(2,1,2);
plot(t, uc_aw, 'Color', [0.8500, 0.3250, 0.0980], 'LineWidth', 2); hold on;
plot(t, u_aw, 'Color', [0.4660, 0.6740, 0.1880], 'LineWidth', 2);
plot(t, u_max*ones(size(t)), 'k--', 'LineWidth', 1);
plot(t, -u_max*ones(size(t)), 'k--', 'LineWidth', 1);
grid on;
set(gca, 'FontSize', 12, 'LineWidth', 1);
title('Control Signals with Anti-Windup');
xlabel('Time (s)');
ylabel('Amplitude');
legend('Unconstrained u_c(t)', 'Saturated u(t)', 'Limits', 'Location', 'best');
```

To further analyze the back-calculation method, the system was simulated with a higher anti-windup gain, $k_{aw} = 10$. The resulting plots are shown in Figure 15.

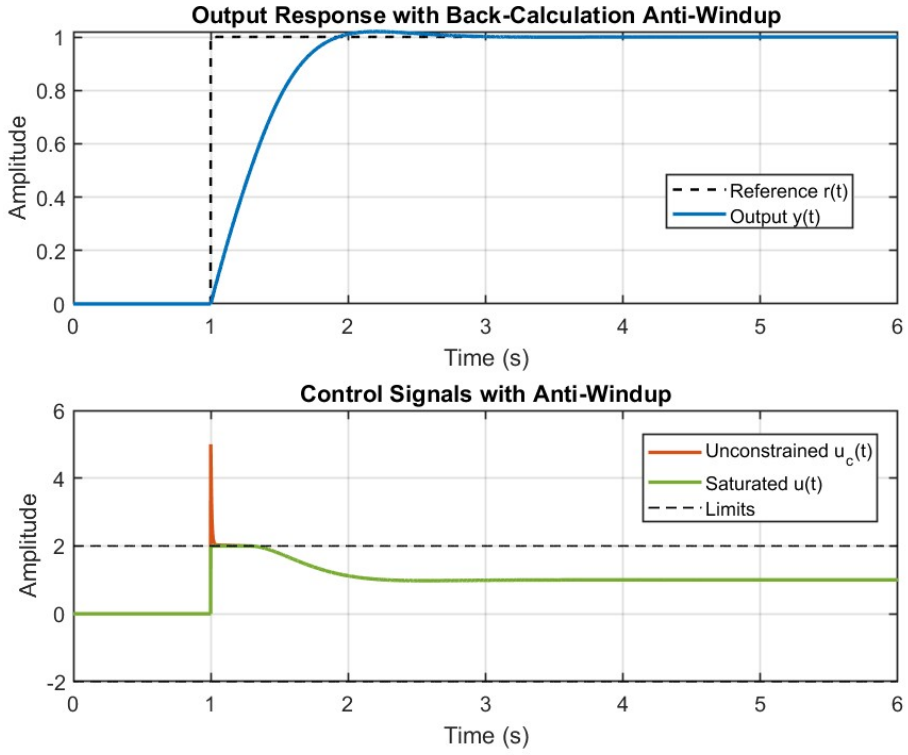


Figure 15: System response and control signals using a PI controller with back-calculation anti-windup ($k_{aw} = 10$).

Effect of Varying k_{aw} on System Performance:

- **Increasing k_{aw} (e.g., $k_{aw} = 10$):** A higher anti-windup gain discharges the integral state more aggressively whenever the actuator saturates. As seen in Figure 15, the unconstrained control signal $u_c(t)$ is forced back to the saturation limit almost instantly after the initial step. This results in an output response $y(t)$ with virtually zero overshoot and a very fast settling time. The controller quickly regains linear operation.
- **Decreasing k_{aw} (e.g., approaching 0):** As k_{aw} decreases, the anti-windup mechanism becomes weaker, and the system behaves more like the original system without anti-windup. The integral state is allowed to accumulate more during saturation, which delays the controller's reaction when the error decreases. This leads to an increase in overshoot and a longer settling time, eventually replicating the severe windup seen in Part (a) if $k_{aw} = 0$.

In summary, increasing k_{aw} provides stronger protection against integral windup, tightening the tracking of the unconstrained control signal to the saturated limits and reducing overshoot, whereas decreasing it allows more windup to occur.

Part (c): Conditional Integration Anti-Windup

In this part, we implement the conditional integration (also known as integrator clamping) anti-windup strategy. The core principle of this method is to halt the integration of the error when the actuator satu-

rates, provided that the current error would push the control signal further into saturation. Specifically, the integrator state derivative $\dot{x}_I$ is governed by the following logic:

$$\dot{x}_I = \{0\ , if(u = u_{max} and e > 0) or (u = -u_{max} and e < 0) e, otherwise$$

This prevents the integral term from accumulating unnecessarily while the plant cannot respond to increased control effort.

To simulate this in MATLAB, a conditional `if-else` statement was introduced within the Euler integration loop. At each time step, the unconstrained control signal $u_c(t)$ is calculated and clamped to the saturation limits ± 2 to obtain the actual control signal $u(t)$. If $u(t)$ reaches the upper limit and the error $e(t)$ is positive, or if it reaches the lower limit and $e(t)$ is negative, the integrator derivative is explicitly set to zero ($\dot{x}_I = 0$). Otherwise, standard integration proceeds ($\dot{x}_I = e$).

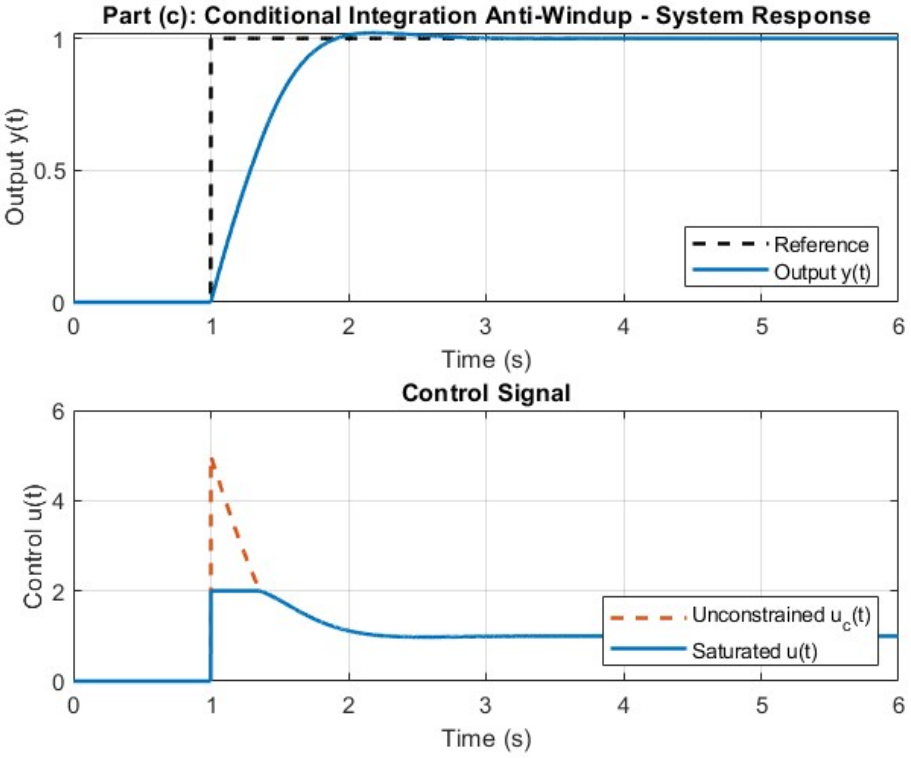


Figure 16: System response and control signals using the conditional integration anti-windup method.

The simulation results are presented in Figure above. As observed in the bottom subplot, when the step input is applied at $t = 1$ s, the unconstrained control signal $u_c(t)$ spikes, but the actual control signal $u(t)$ immediately saturates at the maximum limit of 2. Because the integration is paused during this saturation phase, the unconstrained control signal $u_c(t)$ tracks much closer to the saturated signal, preventing the massive windup seen in the standard PI implementation.

Consequently, as the plant output $y(t)$ approaches the reference, the control signal drops out of saturation almost immediately. Looking at the top subplot, this anti-windup strategy drastically improves the transient performance; the system tracks the reference signal with minimal overshoot and settles rapidly, effectively resolving the performance degradation caused by integral windup.

Part (d): Simulink Implementation of Anti-Windup Methods

To validate the manual numerical implementations developed in the previous sections, the closed-loop system is simulated using Simulink's built-in PID controller block. The objective is to apply both the back-calculation and clamping (conditional integration) anti-windup methods utilizing standard library blocks and compare their transient responses to our manual MATLAB scripts.

The system is constructed in Simulink with a unit step input applied at $t = 1$ s. The plant transfer function is defined as $P(s) = \frac{1}{s+1}$. The control is handled by a standard continuous-time PID Controller block configured for PI action with parallel form gains $K_p = 5$ and $K_i = 15$. To model the actuator limits, the output saturation of the PID block is enabled with upper and lower limits of 2 and -2, respectively. Figure 17 illustrates the block diagram of the closed-loop system along with the internal parameter configuration of the PID controller.

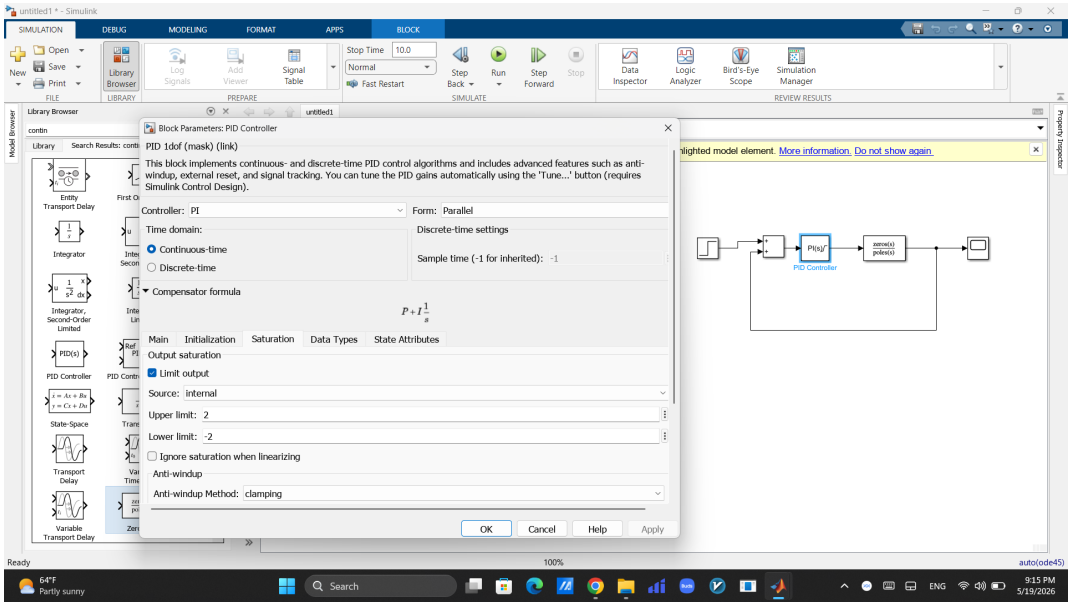


Figure 17: Simulink block diagram and PID controller parameter configuration, demonstrating the saturation limits and anti-windup method selection.

Two separate simulation runs are performed to evaluate the different anti-windup strategies available within the block. First, the anti-windup method is set to "back-calculation". As shown in Figure 18, the system response exhibits a controlled approach to the reference setpoint without the severe overshoot characteristic of standard integral windup. This built-in method successfully computes the difference between the saturated and unsaturated control signals to discharge the integrator, closely matching the manual implementation from Part (b).

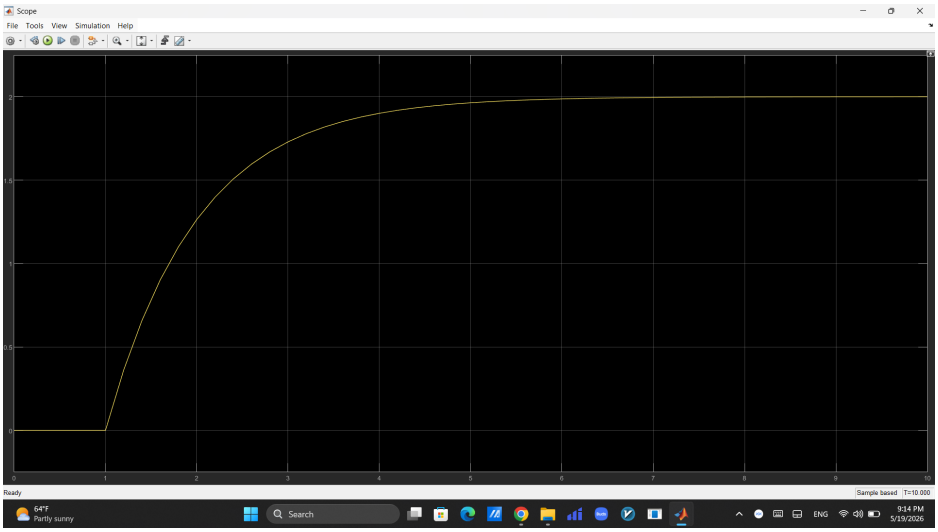


Figure 18: System response $y(t)$ using the Simulink PID block's built-in back-calculation anti-windup method.

Subsequently, the anti-windup method is switched to "clamping". The resulting transient response is shown in Figure 19. The clamping method effectively halts the integration process when the actuator saturates and the control error attempts to drive the output further into saturation. The response is almost identical to the conditional integration approach manually coded in Part (c), demonstrating a swift recovery and zero overshoot. Both simulations confirm that the built-in Simulink algorithms perform the exact same logical and mathematical operations as derived in the earlier parts of this problem.

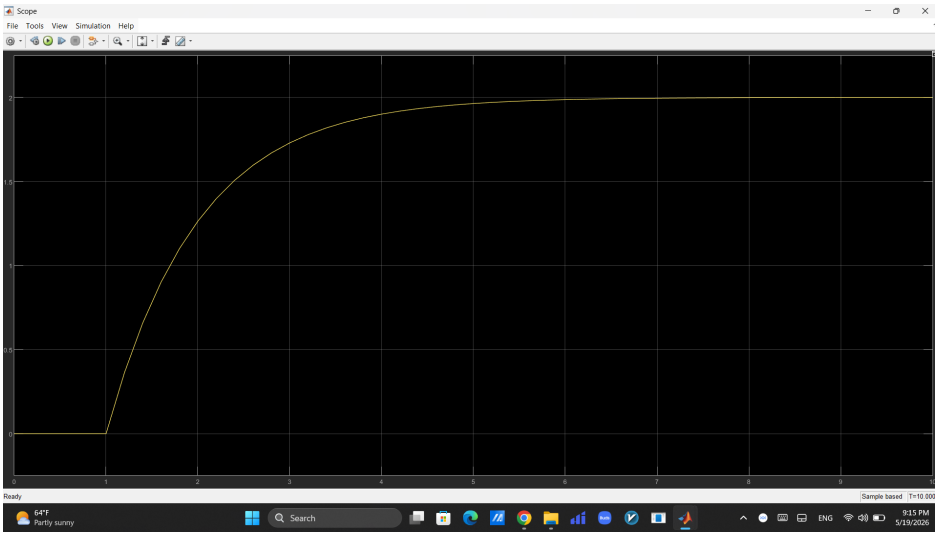


Figure 19: System response $y(t)$ using the Simulink PID block's built-in clamping anti-windup method.

Part (e): Conceptual Analysis

- **Is the windup problem unique to PID controllers, or can it occur in other control strategies?** Integral windup is not unique to PID controllers. It can occur in any control strategy that incorporates internal states driven by the tracking error, particularly integral action. For instance, state-

space controllers augmented with integral states for zero steady-state error, or general dynamic compensators (such as certain lead-lag structures) can also experience windup if the actuator saturates and the internal states continue to integrate the persistent error.

- **Why does increasing the closed-loop bandwidth make windup more severe?**
  Increasing the closed-loop bandwidth generally requires larger controller gains (more aggressive tuning) to achieve faster transient responses. With high gains, even a moderate reference step or disturbance results in a large commanded control signal $u_c(t)$, pushing the actuator into saturation almost immediately. Because the controller is attempting to drive the system faster than the physical actuator limits allow, the sustained error during this period is integrated rapidly, leading to a massive accumulation in the integrator state and exacerbating the windup effect.

- **Why does the integrator state become problematic during saturation?**
  During actuator saturation, the feedback loop is effectively broken; the plant input remains constant at its maximum limit regardless of the controller's computed output. Consequently, the plant output cannot change fast enough to reduce the error. The integrator, unaware of the physical limitation, continues to accumulate this persistent error over time. By the time the controlled variable finally reaches the setpoint and the error changes sign, the integral term is artificially massive. The system must then wait for the negative error to "unwind" this large integral state before the control signal drops below the saturation limit, resulting in significant overshoot and sluggish recovery.

- **How does each anti-windup method modify the integrator dynamics?**
  Both methods aim to manage the integrator state $\dot{x}_I$ during saturation, but they do so via different mechanisms:

  - *Back-calculation* modifies the integrator dynamics by introducing an auxiliary feedback loop. It dynamically subtracts a term proportional to the difference between the saturated and unsaturated control signals from the integrator input ($\dot{x}_I = e + k_{aw}(u - u_c)$). This continuously forces the integrator state to a value consistent with the actuator limits.
  - *Clamping (Conditional Integration)* directly intervenes in the integration process using discrete logic. It completely halts the integrator ($\dot{x}_I = 0$) when the actuator is saturated and the sign of the error is such that it would drive the control signal further into saturation. It resumes normal integration ($\dot{x}_I = e$) only when the error changes sign or the actuator desaturates.

8 Problem 8: System Identification via MATLAB Ident Toolbox

Given the dataset provided in `Ident.mat`, the objective is to employ the MATLAB System Identification (Ident) Toolbox to estimate an optimal transfer function representing the underlying system. Furthermore, this task requires reporting the quantitative indices that characterize the goodness of fit for the estimated model.

The identification process was conducted using the following systematic procedure:

1. **Data Preparation and Import:** The `Ident.mat` file was loaded into the MATLAB workspace. The input signal `u` and the output signal `y` were identified as the primary variables. These were imported into the Ident toolbox, as shown in Figure below, ensuring the correct sample time was established to maintain temporal consistency.

2. **Model Estimation:** Within the Ident environment, the ”Transfer Function” estimation tool was selected. A model structure defined by 2 poles and 1 zero was configured figure to capture the system’s dynamics. The estimation algorithm was then executed to derive the transfer function parameters.

3. **Performance Evaluation:** Upon completion, the model’s accuracy was assessed using the Fit percentage and the Final Prediction Error (FPE) index, as illustrated in the final results window (Figure 23).

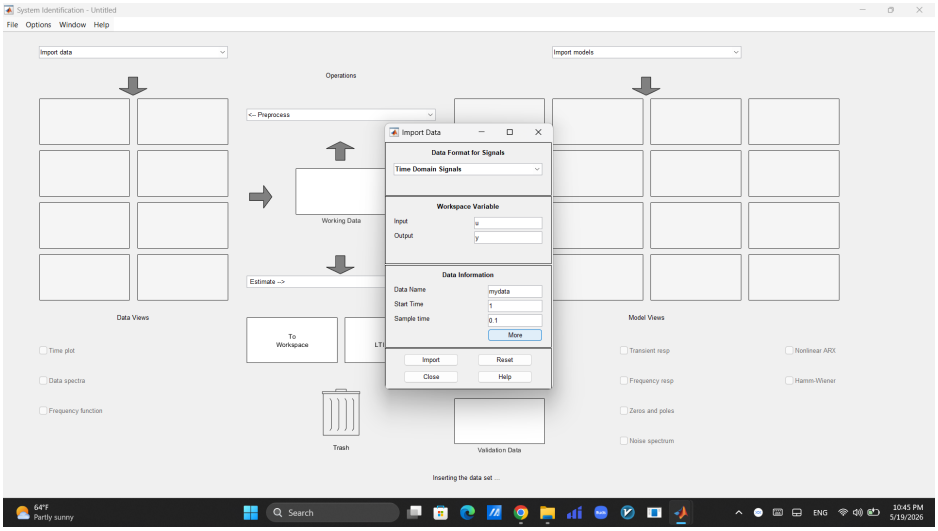


Figure 20: Importing u and y signals into the Ident Toolbox.

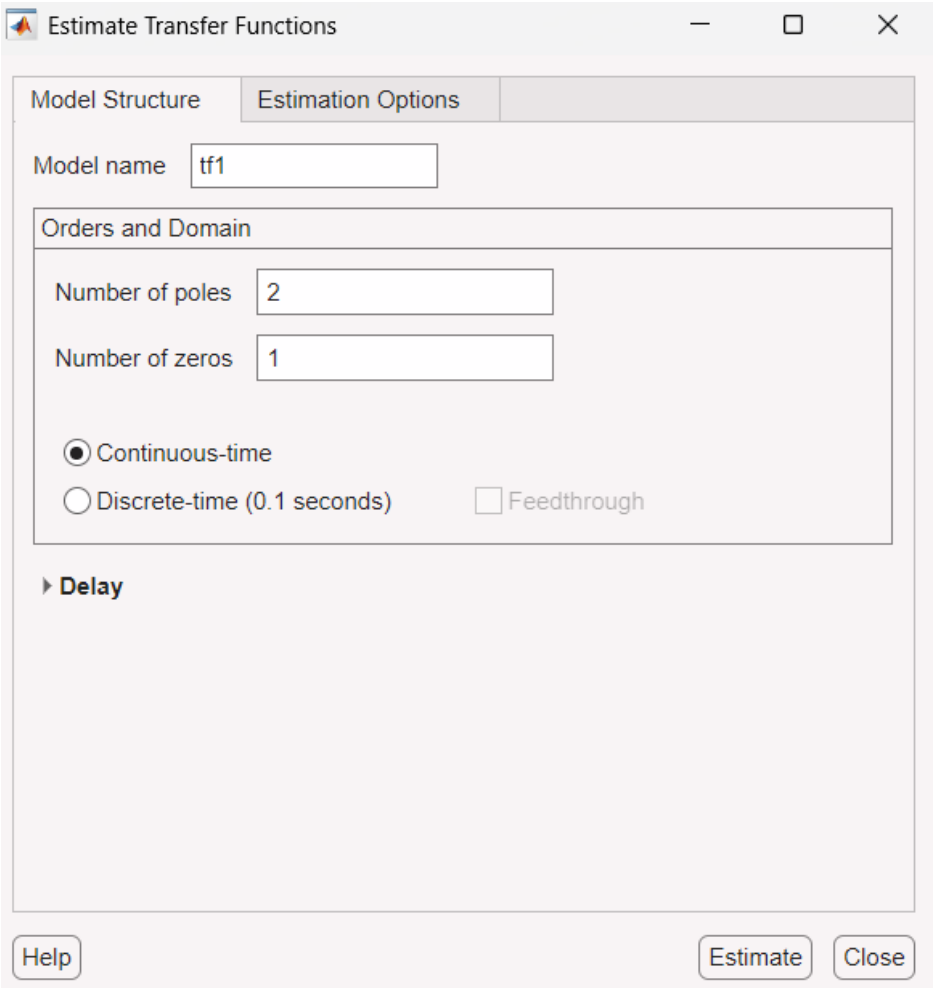


Figure 21: Model structure configuration: Setting poles and zeros.

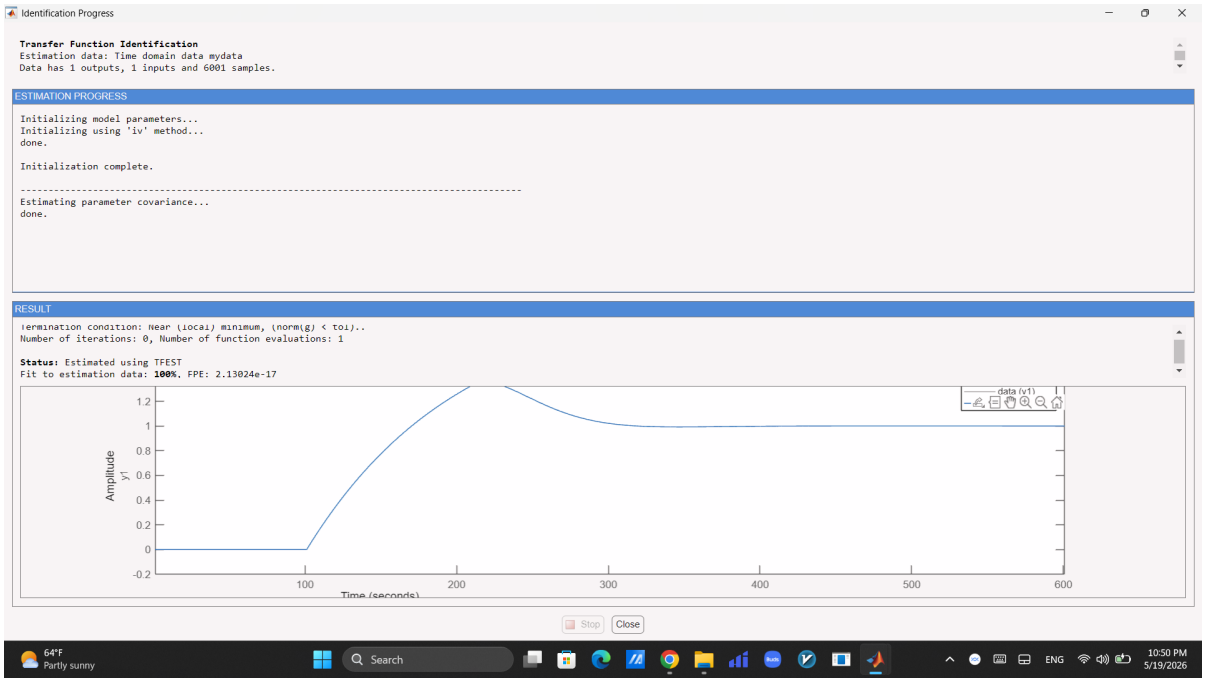


Figure 22: Estimation progress and goodness of fit indices.

The estimation process yielded a highly accurate representation of the system dynamics. The achieved indices are summarized below:

- fit to estimation data: 100%
- **Final prediction error (FPE):** 2.13024×10^{-17}

The perfect fit percentage (100%) and the negligible FPE value indicate that the chosen model structure, with 2 poles and 1 zero, effectively captures the characteristics of the provided dataset.

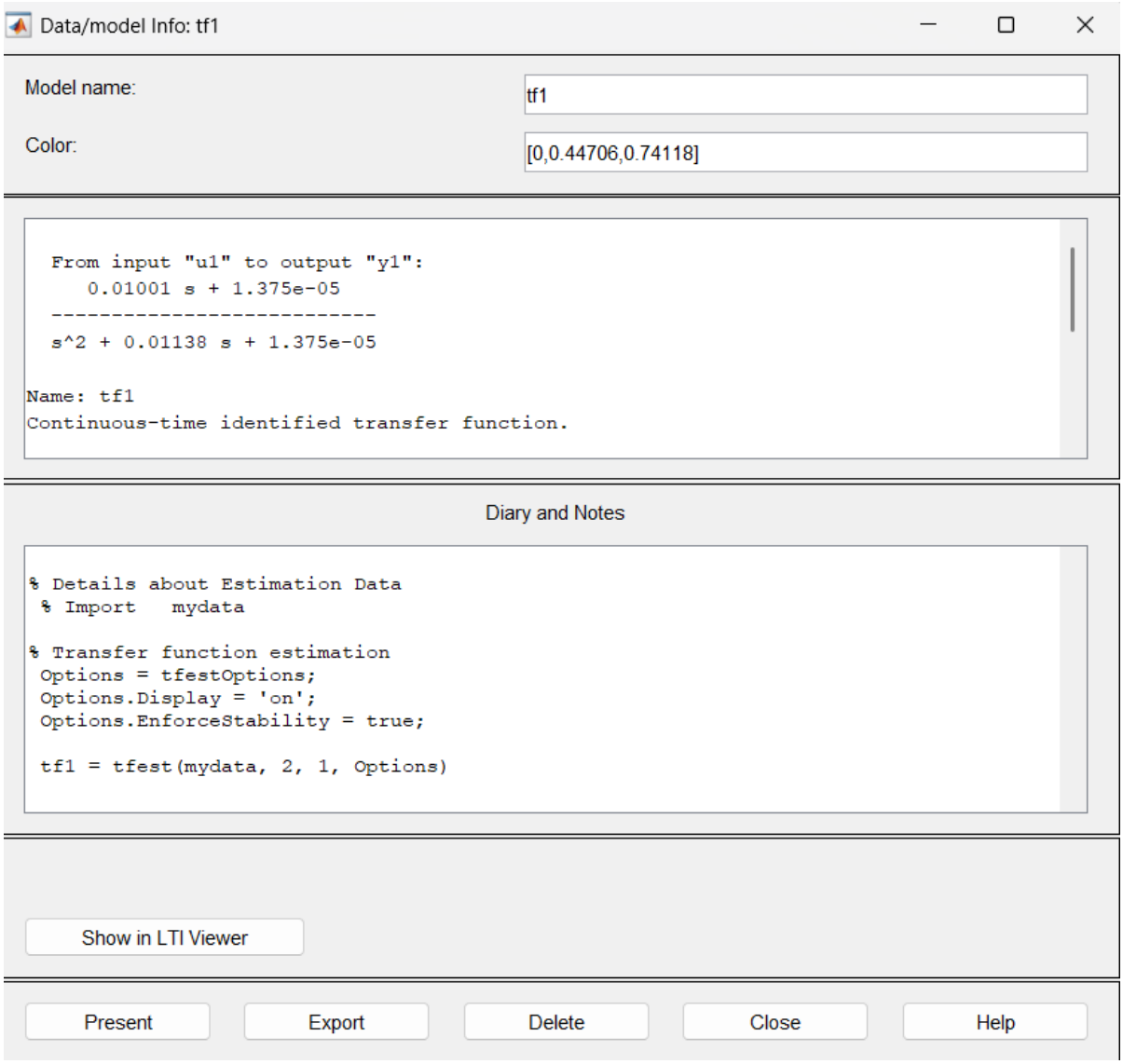


Figure 23: estimated model.

9 problem 9 :

In this exercise, we are tasked with reducing the order of a high-order system using MATLAB's **Model Reducer** app. The original system is a 6th-order continuous-time transfer function given by:

$$G(s) = \frac{(0.2s+1)(s+0.7)(s+1)(s+2)}{(25s+1)(12s+1)(8s+1)(3s+1)(s+0.4)(s+0.2)}$$

The entire reduction session has been saved in the MATLAB workspace file named `ModelReducerSession.mat`.

—

Part 1 & 2: Using Model Reducer App and Mode Selection

First, the transfer function $G(s)$ was defined in the MATLAB command window. Then, the Model Reducer app was launched using the `modelReducer` command. Within the app, the "Mode Selection" method was chosen to truncate the states that contribute the least to the system's overall response.

Based on the DC Contribution of Modal Components bar chart (shown in Figure 24), the first three modes contain the vast majority of the system's energy and DC contribution. Therefore, the system order was reduced from 6 states to 3 states. The cutoff frequencies were adjusted visually in the app to focus on the dominant frequency band.

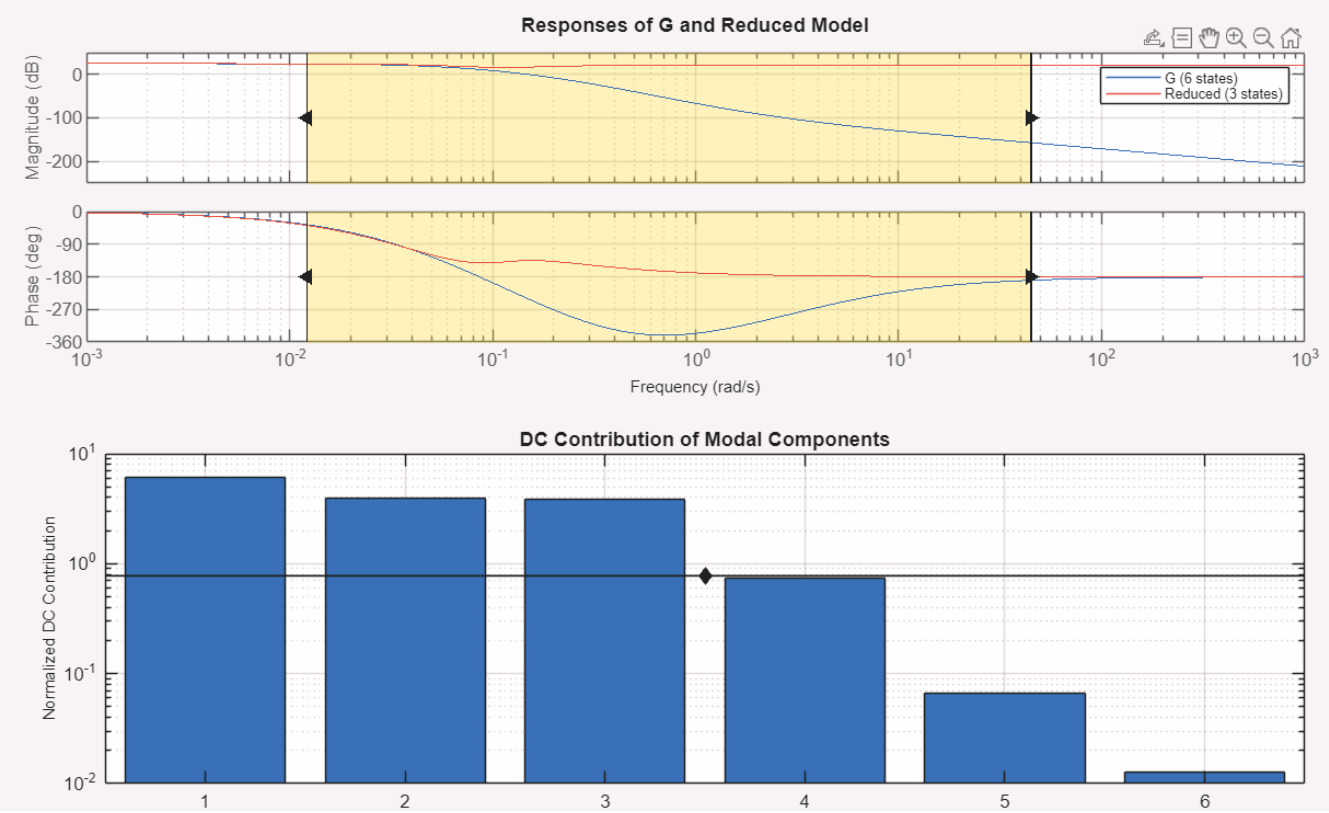


Figure 24: Model Reducer App GUI showing Mode Selection. The top plots show the Bode response highlighting the retained frequency region, and the bottom bar chart shows the DC contribution of the 6 modes, justifying the choice of a 3-state reduced model.

The resulting reduced 3rd-order transfer function was exported to the workspace. Figure shows the command window output detailing the reduced model's continuous-time transfer function in both polynomial and zero/pole/gain formats.

```
ans =

  -0.08507 s^3 + 0.06039 s^2 - 0.02123 s + 0.004746
  ------------------------------------------------
       s^3 + 0.1935 s^2 + 0.013 s + 0.0002712

Continuous-time transfer function.
Model Properties

ans =

  -0.085071 (s-0.4312) (s^2 - 0.2787s + 0.1294)
  ---------------------------------------------
     (s+0.03876) (s^2 + 0.1547s + 0.006998)

Continuous-time zero/pole/gain model.
Model Properties
>>
```

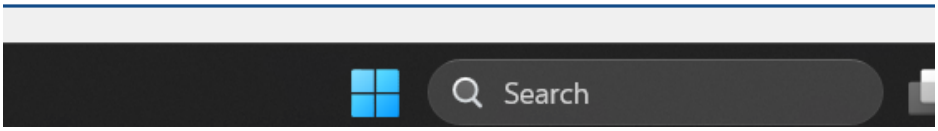


Figure 25: MATLAB command window output showing the mathematical representation of the reduced 3rd-order system.

The reduced transfer function is approximately:

$$G_{reduced}(s) = \frac{-0.08507s^3 + 0.06039s^2 - 0.02123s + 0.004746}{s^3 + 0.1935s^2 + 0.013s + 0.0002712}$$

Part 3: Frequency Response Comparison

A Bode plot was generated in MATLAB to compare the original 6th-order system and the reduced 3rd-order system.

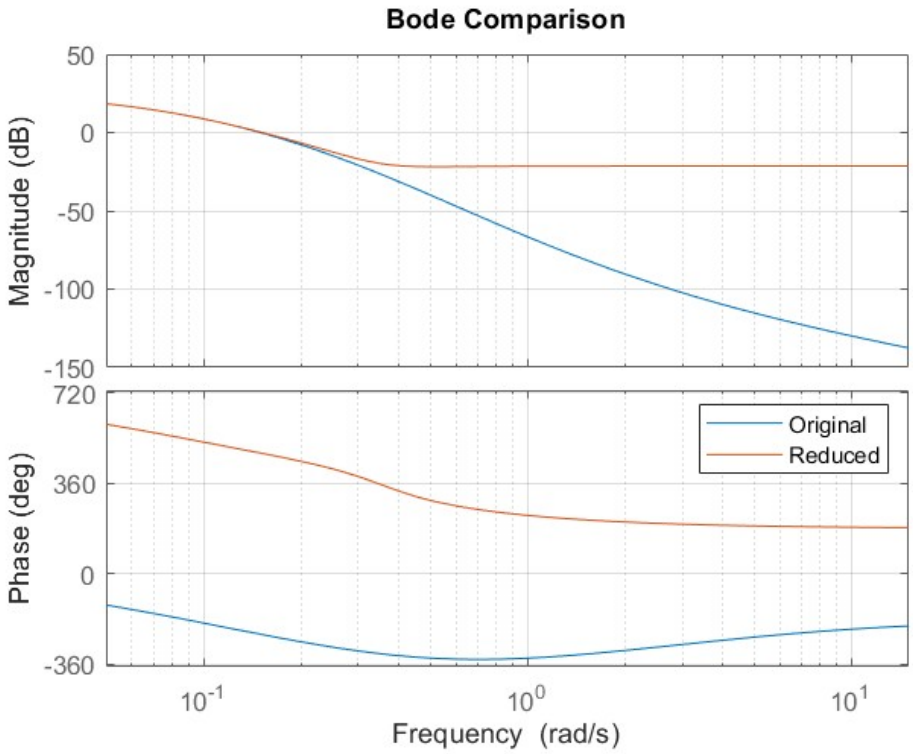


Figure 26: Bode comparison between the original 6th-order model and the reduced 3rd-order model.

As seen in Figure above, within the low-frequency range, the magnitude of the reduced model closely tracks the original model. However, as the frequency increases towards $\omega = 15$ rad/s, the magnitude of the original system continues to roll off, while the reduced model levels out. The phase plot shows a significant divergence starting around $\omega = 0.1$ rad/s.

Part 4: Performance Analysis and Loss of Accuracy

The reduced model successfully captures the dominant low-frequency dynamics and the steady-state behavior (DC gain) of the original system, which is typically the most critical region for control system design and stability analysis. By retaining the 3 most significant modes, we simplify the computational complexity significantly.

loss of accuracy: The truncation of the 3 higher-frequency modes results in a noticeable loss of accuracy at higher frequencies (specifically for $\omega > 0.1$ rad/s).

- **Magnitude:** The original system has a higher relative degree (more poles than zeros), causing its high-frequency magnitude to continuously decrease with a steep slope. The reduced model, having an equal number of poles and zeros (order 3), exhibits a flattened magnitude response at high frequencies, leading to a steady error.
- **Phase:** The phase deviation is severe at higher frequencies. The original system's phase drops significantly due to the accumulation of phase lag from its 6 poles. The reduced model cannot replicate this extensive phase lag with only 3 poles.

In conclusion, the reduced model is an excellent approximation for low-frequency inputs and steady-state analysis but is inadequate if the system is subjected to high-frequency excitations or if high-frequency transient fidelity is strictly required.

10 Problem 10: Curve Fitting Toolbox Task

In this task, the MATLAB Curve Fitting Toolbox was employed to analyze several datasets representing biological growth processes over time. The objective was to fit each dataset to four well-known nonlinear growth models and evaluate their effectiveness using quantitative goodness-of-fit metrics. The models considered were the Logistic, Gompertz, Richards (generalized logistic), and Bertalanffy growth models, defined respectively as:

$$\textbf{Logistic:} \quad P(t) = \frac{K}{1 + \frac{K - P_0}{P_0} e^{-rt}}$$

$$\textbf{Gompertz:} \quad P(t) = K \exp\left(-e^{-r(t-t_0)}\right)$$

$$\textbf{Richards:} \quad P(t) = \frac{K}{\left(1 + \alpha e^{-rt}\right)^{1/\nu}}$$

$$\textbf{Bertalanffy:} \quad P(t) = K\left(1 - e^{-rt}\right)^3$$

These models describe sigmoidal growth behavior and differ in flexibility and structural complexity.

(a) Data Loading

The datasets `GrowthData1.csv` through `GrowthData4.csv` were imported into MATLAB using `readtable`. Each dataset contained two variables: time t (independent variable) and measured growth $P(t)$ (dependent variable).

The following MATLAB commands were used:

```
T1 = readtable('GrowthData1.csv');
T2 = readtable('GrowthData2.csv');
T3 = readtable('GrowthData3.csv');
T4 = readtable('GrowthData4.csv');

t1 = T1.Var1;  y1 = T1.Var2;
t2 = T2.Var1;  y2 = T2.Var2;
t3 = T3.Var1;  y3 = T3.Var2;
```

```
t4 = T4.Var1;  y4 = T4.Var2;
```

Initial inspection confirmed that the datasets exhibit bounded growth behavior characteristic of biological systems approaching a saturation level.

(b) Model Fitting Using Curve Fitting Toolbox

The Curve Fitting Toolbox was launched using:

```
cftool
```

For each dataset:

- The corresponding (t_k, y_k) variables were selected.
- The fit type was changed to **Custom Equation**.
- Each of the four theoretical models was entered explicitly.
- Nonlinear least-squares optimization was performed automatically.
- Fit results were saved and exported.

The custom equations entered were:

```
Logistic:
y = K./(1 + ((K-P0)./P0).*exp(-r*x))

Gompertz:
y = K.*exp(-exp(-r*(x-t0)))

Richards:
y = K./((1 + alpha.*exp(-r*x)).^(1/nu))

Bertalanffy:
y = K.*(1-exp(-r*x)).^3
```

The toolbox estimated parameters using nonlinear regression and provided statistical metrics including SSE, R^2, adjusted R^2, and RMSE.

(c) Plots and Accuracy Discussion

The fitted curves for Dataset 1 are illustrated below.

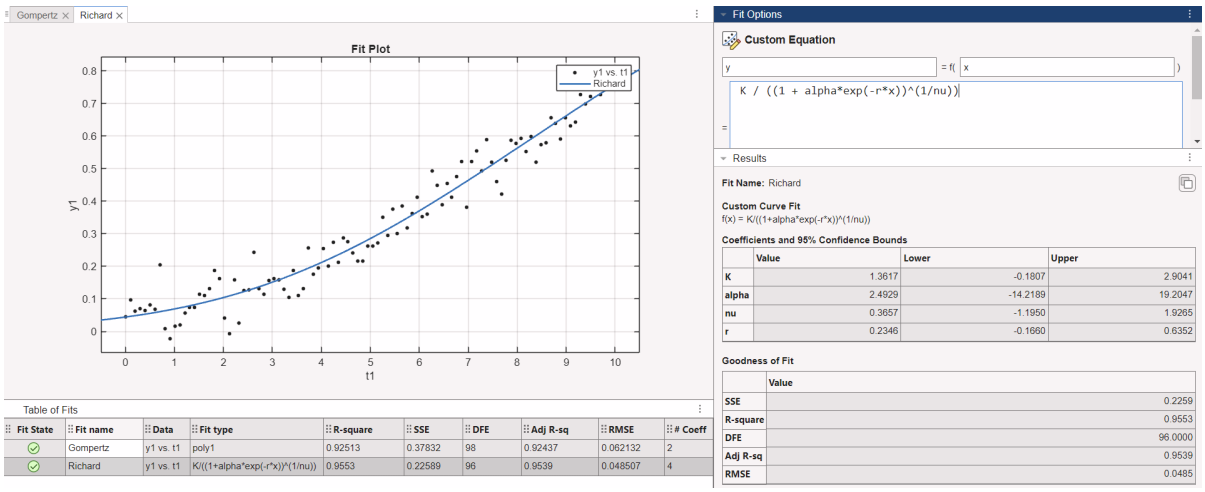


Figure 27: Richards model fit

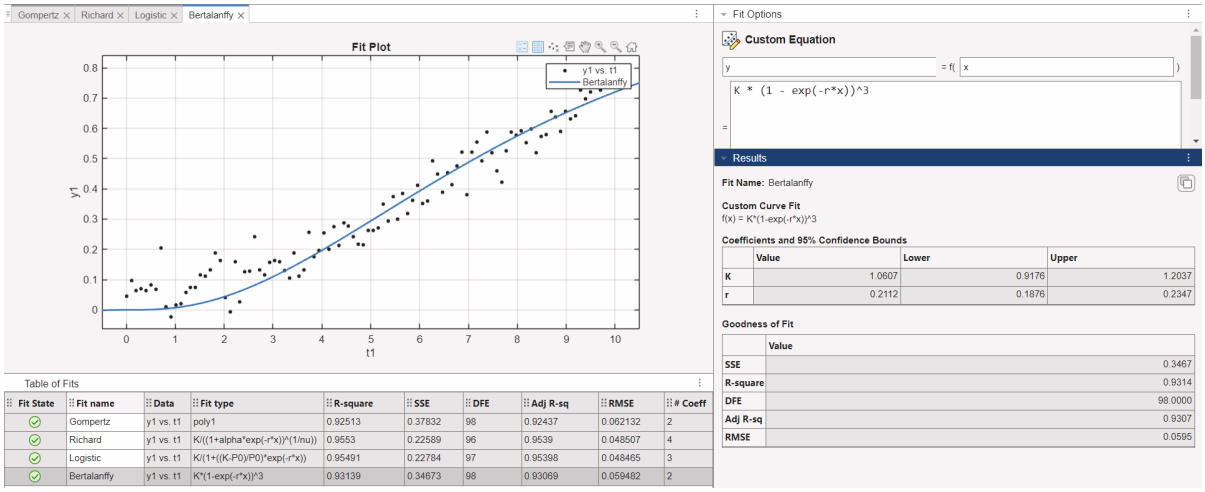


Figure 28: Bertalanffy model fit

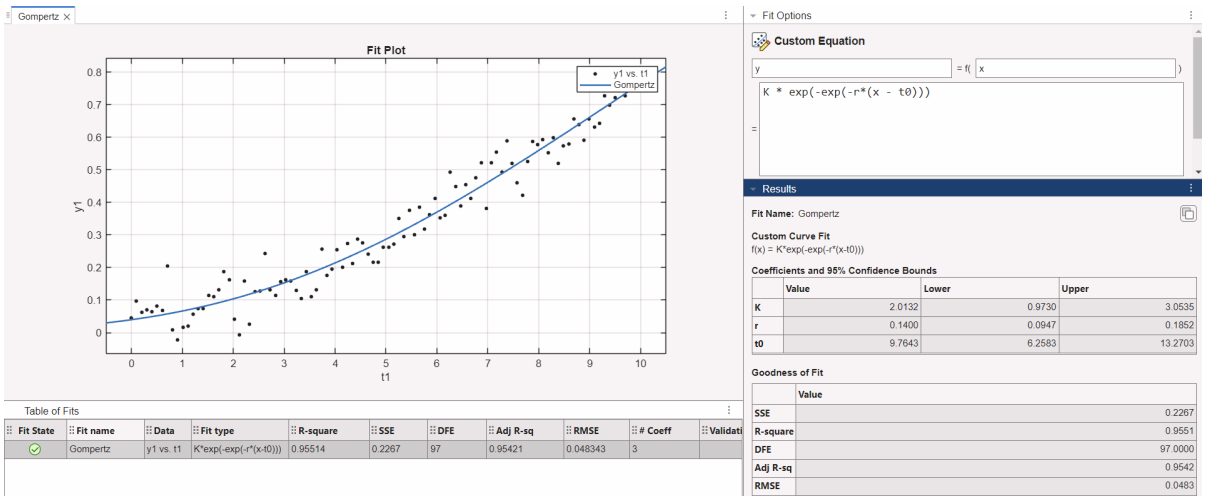


Figure 29: Gompertz model fit

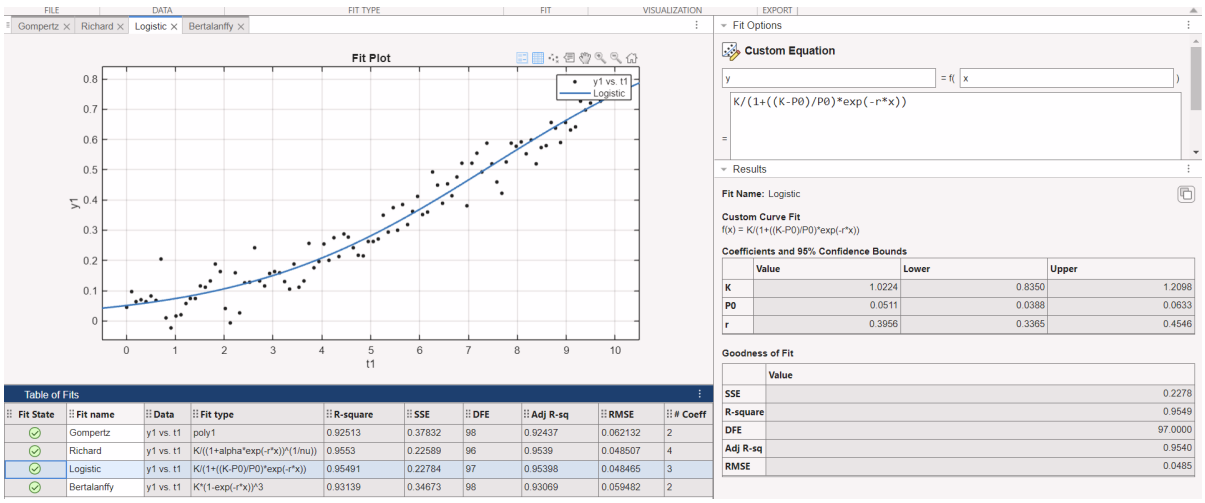


Figure 30: Logistic model fit

All models successfully reproduce the sigmoidal growth behavior. However, visual inspection reveals that the Richards and Logistic models align more closely with the experimental data, especially near the inflection point and saturation region. The Bertalanffy and Gompertz models show slightly larger deviations in the mid-growth phase.

(d) Goodness-of-Fit Comparison

The quantitative comparison for Dataset 1 is summarized below:

Model	R^2	SSE	RMSE
Richards	0.9553	0.22589	0.048507
Logistic	0.95491	0.22784	0.048465
Bertalanffy	0.93139	0.34673	0.059482
Gompertz	0.92513	0.37832	0.062132

The Richards model achieves the highest R^2 value and one of the lowest RMSE values, indicating superior explanatory power. The Logistic model performs nearly identically, while Bertalanffy and Gompertz demonstrate comparatively larger residual errors.

(e) Model Selection and Theoretical Analysis

The Logistic and Richards models provide the most accurate representation of the biological growth behavior. The primary reason is structural flexibility:

- The **Logistic model** assumes symmetric sigmoidal growth and captures saturation dynamics effectively.
- The **Richards model** generalizes the Logistic model by introducing the parameter ν, allowing asymmetry control and improved flexibility.
- The **Gompertz model** is suitable for asymmetric growth but may underestimate late-stage saturation accuracy.
- The **Bertalanffy model** is structurally constrained and best suited for growth governed by surface-area mechanisms.

From both theoretical reasoning and statistical evidence, the **Richards model is the most appropriate** for Dataset 1 due to its highest R^2 and lowest residual error, while maintaining biological interpretability. The Logistic model remains a strong alternative when model simplicity is preferred.

Overall, the Curve Fitting Toolbox provides a systematic and robust framework for nonlinear biological growth modeling, combining visual diagnostics with rigorous statistical validation.

11 problem 11

11.1 Part a :

Assuming a standard unity negative feedback control architecture, the closed-loop transfer function $T(s)$ relating the reference input $R(s)$ to the system output $Y(s)$ is given by:

$$T(s) = \frac{C(s)G(s)}{1 + C(s)G(s)} \tag{1}$$

The PIDF controller $C(s)$ and the chemical reactor plant $G(s)$ are defined as: C(s) = $K_p + \frac{K_i}{s} + \frac{K_d s}{T_f s+1}$ $G(s) = \frac{2e^{-0.3s}}{(4s+1)(3s+1)}$ To simplify the derivation, we first express the controller $C(s)$ over a common denominator:

$$C(s) = \frac{K_p s(T_f s+1) + K_i(T_f s+1) + K_d s^2}{s(T_f s+1)} = \frac{(K_p T_f + K_d)s^2 + (K_p + K_i T_f)s + K_i}{s(T_f s+1)} \tag{2}$$

Multiplying $C(s)$ and $G(s)$ gives the open-loop transfer function $L(s)$:

$$L(s) = \frac{2e^{-0.3s}\left[(K_pT_f + K_d)s^2 + (K_p + K_iT_f)s + K_i\right]}{s(T_fs+1)(4s+1)(3s+1)} \tag{3}$$

Substituting $L(s)$ into the closed-loop formula $T(s) = \frac{L(s)}{1+L(s)}$ yields the overall closed-loop transfer function:

$$T(s) = \frac{2e^{-0.3s}\left[(K_pT_f + K_d)s^2 + (K_p + K_iT_f)s + K_i\right]}{s(T_fs+1)(4s+1)(3s+1) + 2e^{-0.3s}\left[(K_pT_f + K_d)s^2 + (K_p + K_iT_f)s + K_i\right]} \tag{4}$$

Role of the Filter Term in the Derivative Component:

An ideal derivative controller term, represented by K_ds, has a magnitude that increases linearly with frequency. This characteristic causes the controller to severely amplify high-frequency measurement noise, leading to aggressive and potentially damaging control efforts (actuator chattering).

By introducing the first-order low-pass filter denominator (T_fs+1), the derivative term becomes proper: $\frac{K_ds}{T_fs+1}$.

- At low frequencies ($\omega \ll 1/T_f$), the term behaves like an ideal derivative, providing the necessary phase lead to improve the system's transient response.
- At high frequencies ($\omega \gg 1/T_f$), the gain of the derivative term asymptotically bounds to a constant value of K_d/T_f. This rolls off the high-frequency gain, effectively limiting noise amplification while preserving system stability and robustness.

11.2 b :

The closed-loop stability of the system is governed by the roots of the characteristic equation, $1 + C(s)G(s) = 0$. Substituting the transfer functions, we obtain:

$$s(T_fs+1)(4s+1)(3s+1) + 2e^{-0.3s}\left[(K_pT_f + K_d)s^2 + (K_p + K_iT_f)s + K_i\right] = 0 \tag{5}$$

The presence of the exponential delay term $e^{-0.3s}$ makes the system infinite-dimensional, yielding an infinite number of poles. To analytically determine the constraints on the PIDF parameters (K_p, K_i, K_d, T_f) using classical methods like the Routh-Hurwitz criterion, we must approximate the delay using a first-order Padé approximation (i used my slides from my Linear Contro course i had with Prof. Ahi for this) :

$$e^{-0.3s} \approx \frac{1 - 0.15s}{1 + 0.15s} \tag{6}$$

Substituting this approximation into the characteristic equation yields a strictly rational polynomial:

$$s(T_fs+1)(12s^2+7s+1)(1+0.15s) + 2(1-0.15s)\left[(K_pT_f + K_d)s^2 + (K_p + K_iT_f)s + K_i\right] = 0 \tag{7}$$

Expanding this gives a fifth-order polynomial characteristic equation of the form $a_5s^5 + a_4s^4 + a_3s^3 + a_2s^2 + a_1s + a_0 = 0$. For asymptotic stability, the necessary constraints require all coefficients to be

strictly positive ($a_i > 0$). At a minimum, this imposes the fundamental requirements:

$$K_p > 0, \quad K_i > 0, \quad K_d > 0, \quad T_f > 0 \tag{8}$$

Furthermore, the non-minimum phase zero introduced by the Padé approximation ($(1-0.15s)$) explicitly limits the maximum allowable controller gains. Specifically, the constant term $a_0 = 2K_i$ requires $K_i > 0$, while the s term a_1 will contain negative contributions from the delay, establishing an upper theoretical bound on the integral gain K_i and proportional gain K_p to prevent the roots from crossing into the right-half s-plane. Exact upper constraints are found by formulating the Routh array and ensuring all terms in the first column remain strictly positive. which again is competley a linear control question and not too hard .

11.3 C :

In heuristic optimization (such as a Genetic Algorithm), the choice of the cost (or objective) function dictates the behavior of the resulting tuned controller. Three primary cost functions are compared below:

1. Integral of Absolute Error (IAE):

$$IAE = \int_0^\infty |e(t)|dt \tag{9}$$

IAE treats all errors equally regardless of when they occur or their magnitude. *Influence:* Optimizing for IAE generally produces a balanced response. It minimizes the overall error without over-penalizing the large initial error (unlike ISE), resulting in a response with moderate overshoot and moderate settling time. It is a good choice for general-purpose loop tuning where steady-state error and transient performance are equally important.

2. Integral of Squared Error (ISE):

$$ISE = \int_0^\infty e^2(t)dt \tag{10}$$

ISE squares the error magnitude, meaning large errors are penalized heavily, while small errors (less than 1) contribute very little to the total cost. *Influence:* Because the initial transient error is typically the largest, an ISE-optimized controller will act aggressively to reduce the error quickly, often leading to a fast rise time. However, because small persistent errors are barely penalized, the resulting system may exhibit higher overshoot and prolonged low-amplitude oscillations (ringing) before finally settling.

3. Integral of Time-Weighted Absolute Error (ITAE):

$$ITAE = \int_0^\infty t|e(t)|dt \tag{11}$$

ITAE introduces a time multiplier to the absolute error. Errors that occur early in the response (when t is small) are lightly penalized, while errors that persist later in time (when t is large) are heavily penalized. *Influence:* An ITAE-optimized PIDF controller prioritizes bringing the system to its final steady-state quickly and smoothly. It heavily discourages prolonged oscillations and steady-state error. Consequently, ITAE generally produces the most robust and highly damped responses, characterized by minimal overshoot and the shortest overall settling time compared to IAE and ISE.

4. Integral of Time-Weighted Squared Error (ITSE):

$$ITSE = \int_0^\infty te^2(t)dt \tag{12}$$

Application & Effect: ITSE combines the time-weighting characteristic of ITAE with the squared error penalty of ISE. In an optimization algorithm, this objective function severely penalizes large errors that occur late in the time response. Consequently, an ITSE-optimized controller will aggressively suppress sustained oscillations. This typically results in a very smooth transient response with excellent steady-state tracking and rapid settling, though it might exhibit a slightly slower initial rise time compared to pure ISE.

5. Root Mean Square Error (RMSE):

$$RMSE = \sqrt{\frac{1}{T}\int_0^T e^2(t)dt} \tag{13}$$

Application & Effect: RMSE provides a time-averaged, normalized metric of the squared error over a specific observation window T. In the optimization process, it acts similarly to ISE by forcing the controller to quickly eliminate the large initial errors, promoting a fast transient response (quick rise time). Because it averages the error, it balances the penalty between the initial transient surge and the ongoing steady-state error. However, like ISE, it may tolerate small, persistent oscillations since tiny errors contribute negligibly to the overall square root of the mean.

6. Maximum Overshoot (MPO):

$$MPO = \max\left(\frac{|y_{max} - y_{ss}|}{y_{ss}} \times 100\%\right) \tag{14}$$

Application & Effect: MPO focuses exclusively on the peak transient deviation. If applied as the sole cost function in an optimization routine, the algorithm would likely generate an overly sluggish, overdamped response because it would ignore rise time, settling time, and steady-state error entirely in its singular pursuit of zero overshoot. Therefore, in practical optimization, MPO is usually applied either as a strict penalty/constraint (e.g., rejecting any controller parameters that result in $MPO > 10\%$) or combined into a multi-objective cost function. When used this way, it strictly bounds the transient aggressiveness while allowing other metrics (like IAE or ITAE) to optimize the steady-state tracking and speed.

11.4 d :

Classical PID tuning methods such as Ziegler–Nichols (ZN) and Cohen–Coon (CC) determine controller parameters using analytical formulas derived from simplified plant models or experimental response data. In contrast, a Genetic Algorithm (GA) treats PID tuning as a numerical optimization problem, where the controller parameters (K_p, K_i, K_d, T_f) are iteratively evolved to minimize a selected performance index.

The principal advantage of GA-based tuning is its flexibility. Unlike ZN and CC, which rely on low-order approximations and assume relatively simple plant dynamics, a GA directly optimizes the actual

closed-loop response. This is especially important for the present chemical reactor system:

$$G(s) = \frac{2e^{-0.3s}}{(4s+1)(3s+1)} \tag{15}$$

because the transport delay term $e^{-0.3s}$ introduces nontrivial dynamics and significantly limits the accuracy of classical tuning correlations.

A second major advantage is that a GA supports arbitrary objective functions. From Part (c), several performance indices were defined:

$$IAE, \quad ISE, \quad ITAE, \quad ITSE, \quad RMSE, \quad MPO \tag{16}$$

A GA can optimize a weighted combination of these objectives simultaneously. Consequently, the designer can explicitly enforce engineering requirements such as:

- low overshoot,
- fast rise time,
- low steady-state error,
- bounded control effort,
- robustness against oscillation.

Classical methods do not directly optimize these quantities. For example, the standard ZN tuning rules intentionally produce an aggressive response with approximately 25% overshoot. Such behavior violates the design requirement of less than 5% overshoot.

Another advantage is that GA optimization does not require gradient information or a differentiable model. Therefore, nonlinearities, actuator saturation, time delays, and discontinuous penalty functions can all be incorporated naturally into the optimization process.

However, GA tuning also has several limitations.

First, the computational cost is substantially higher than classical methods. ZN or CC produce controller parameters almost immediately from a few measurements, whereas a GA requires repeated simulation of the closed-loop system over many generations and candidate solutions.

Second, convergence to the global optimum is not guaranteed. The final solution depends on:

- population size,
- mutation rate,
- crossover probability,
- initialization range,
- stopping criterion.

Poor parameter selection can produce premature convergence or unstable controllers.

Third, GA-based tuning lacks the analytical transparency of classical methods. ZN and CC provide direct physical insight into the relationship between plant dynamics and controller gains, while GA optimization behaves largely as a black-box search procedure.

Finally, classical methods remain attractive for industrial implementation because of their simplicity and speed. For stable low-order plants without severe delay or constraints, ZN and CC often provide sufficiently good initial tuning with minimal computational effort.

Therefore, classical methods are useful for rapid preliminary tuning, while genetic algorithms are more suitable for constrained multi-objective optimization problems involving delayed or complex systems such as the present reactor process.

11.5 e :

From Part (c), different performance indices emphasize different aspects of the transient response:

- *ISE* and *RMSE* favor rapid error reduction and therefore reduce rise time.
- *ITAE* strongly penalizes persistent oscillations and steady-state error.
- *MPO* directly constrains overshoot.

The present problem is inherently a constrained multi-objective optimization problem. Therefore, the cost function must combine:

1. transient-speed objectives,
2. steady-state accuracy objectives,
3. actuator constraint penalties,
4. control-energy minimization.

Let: e(t) = r(t)-y(t)
u(t) = controller output
$M_p = maximumovershoot(\%)$
$t_r = risetime$

A suitable composite objective function is:

$$J = w_1 \int_0^T e^2(t)\,dt + w_2 \int_0^T t|e(t)|\,dt + w_3 t_r + w_4 \int_0^T u^2(t)\,dt + P_{OS} + P_{SS} + P_u \tag{17}$$

where:

$$w_1, w_2, w_3, w_4 > 0 \tag{18}$$

are weighting coefficients selected according to the desired tradeoff.

Each term has a specific role.

The first term,

$$\int_0^T e^2(t)\,dt \tag{19}$$

is the *ISE* criterion from Part (c). Since large errors are squared, this term aggressively penalizes the initial transient error and therefore reduces rise time.

The second term,

$$\int_0^T t|e(t)|\,dt \tag{20}$$

is the *ITAE* criterion from Part (c). Because the error is multiplied by time, oscillations and slow settling are heavily penalized. This improves damping and steady-state convergence.

The third term,

$$t_r \tag{21}$$

directly penalizes slow rise time. Including t_r explicitly prevents the optimizer from producing an excessively sluggish overdamped response.

The fourth term,

$$\int_0^T u^2(t)\,dt \tag{22}$$

penalizes excessive control effort. This prevents unrealistically large actuator activity and improves practical implementability.

The remaining terms are constraint penalties.

To enforce the overshoot specification $M_p < 5\%$, define:

$$P_{OS} = \lambda_1 \max(0, M_p - 5)^2 \tag{23}$$

where $\lambda_1 \gg 1$ is a large penalty constant. If the overshoot remains below 5%, this term becomes zero. Otherwise, the cost increases quadratically.

To guarantee steady-state accuracy:

$$P_{SS} = \lambda_2 \max\left(0, |e_{ss}| - 0.01\right)^2 \tag{24}$$

where:

$$e_{ss} = \lim_{t\to\infty} e(t) \tag{25}$$

This term forces the optimizer to satisfy the requirement:

$$|e_{ss}| < 1\%$$

Finally, the actuator constraint:

$$-5 \le u(t) \le 5$$

is enforced through:

$$P_u = \lambda_3 \int_0^T \left[\max(0, |u(t)| - 5)\right]^2 dt \tag{26}$$

If the control signal remains inside the allowable range, the penalty is zero. Any saturation violation produces a rapidly increasing cost.

Therefore, the final optimization problem becomes:

$$\min_{K_p,K_i,K_d,T_f} J \tag{27}$$

subject to the stability constraints derived in Part (b):

$$K_p > 0, \quad K_i > 0, \quad K_d > 0, \quad T_f > 0 \tag{28}$$

This cost function is suitable for Genetic Algorithm optimization because it simultaneously incorporates:

- fast transient response,
- low overshoot,
- low steady-state error,
- bounded actuator behavior,
- reduced control energy.

Moreover, the structure directly follows the behavioral properties of *ISE*, *ITAE*, and *MPO* established previously in Part (c).

11.6 f and g :

Tuning a Proportional-Integral-Derivative (PID) controller for systems with transport delay (dead time) is a notoriously challenging problem in classical control theory. The plant under consideration is given by the transfer function:

$$G(s) = \frac{2}{(5s+1)(2s+1)} e^{-0.2s}$$

The presence of the time delay, $e^{-0.2s}$, introduces a continuous phase lag that increases with frequency, significantly reducing the phase margin and making the closed-loop system prone to instability and oscillatory behavior. Standard tuning rules (like Ziegler-Nichols) often yield highly aggressive controllers for delayed plants. Therefore, formulating the tuning process as a constrained optimization problem is a far more robust approach.

The objective is to find the optimal PID parameter vector, $\mathbf{K} = [K_p, K_i, K_d]^T$, that minimizes the Integral of Time-weighted Absolute Error (ITAE). The ITAE criterion is mathematically defined as:

$$J_{ITAE}(\mathbf{K}) = \int_0^{t_f} t|e(t)|dt$$

where $e(t) = r(t) - y(t)$ is the error between the reference step input $r(t)$ and the plant output $y(t)$. ITAE is particularly well-suited for this application because the time-weighting multiplier (t) heavily penalizes long-duration transients and steady-state errors, promoting a fast settling time.

design of the Custom Objective Function

To enforce the strict design requirements without relying on the optimization algorithms' built-in constraint handlers (which can sometimes fail on complex simulation-based boundaries), we constructed a custom cost function that evaluates the closed-loop step response and applies massive scalar penalties.

The composite cost function evaluated by the optimizers is:

$$J_{total} = J_{ITAE} + P_{OS} + P_{SS} + P_U$$

Where the penalty terms are defined as follows:

- **Overshoot Penalty (P_{OS}):** If the maximum percentage overshoot exceeds 5%, a massive penalty (e.g., 10^5) is added.
- **Steady-State Error Penalty (P_{SS}):** If the absolute steady-state error $|e(t_f)|$ exceeds 1%, another heavy penalty is added.
- **Control Effort Penalty (P_U):** Practical actuators have saturation limits. If the control signal $u(t)$ exceeds the bounds of $[-5, 5]$ at any point in time, a penalty is applied to discard that solution.

optimization

To find the global minimum of this complex cost landscape, we compared three distinct optimization paradigms available in MATLAB:

1. **Genetic Algorithm ('ga'):** A population-based heuristic algorithm inspired by natural selection. It utilizes mechanisms such as crossover (combining traits of good controllers) and mutation (randomly altering parameters) to explore the search space and escape local minima.
2. **Particle Swarm Optimization ('particleswarm'):** A swarm intelligence algorithm modeling flocking behavior. Search agents (particles) move through the parameter space, adjusting their velocities based on their own best discovered positions and the global best position found by the entire swarm.
3. **Non-linear Programming ('fmincon'):** A local, gradient-based optimization solver (typically using Sequential Quadratic Programming or Interior-Point methods). It relies on evaluating the local gradient of the cost function to step towards a minimum.

The search space was bounded, and the algorithms were executed. The command window output tracking the convergence behavior is provided below:

```
Running Genetic Algorithm (ga)...

Single objective optimization:
3 Variables

Options:
```

```
CreationFcn:       @gacreationuniform
CrossoverFcn:      @crossoverscattered
SelectionFcn:      @selectionstochunif
MutationFcn:       @mutationadaptfeasible

                                 Best           Mean      Stall
Generation      Func-count        f(x)           f(x)    Generations
    1              100       4.804e+05       8.454e+10        0
... [Output truncated for brevity] ...
   50             2403           11.04            4083        0
ga stopped because it exceeded options.MaxGenerations.

Running Particle Swarm Optimization...

                                 Best           Mean     Stall
Iteration     f-count            f(x)           f(x)    Iterations
    0              50       7.349e+05       2.421e+10        0
... [Output truncated for brevity] ...
   50             2550           11.12       3.506e+04        6
Optimization ended: number of iterations exceeded OPTIONS.MaxIterations.

Running fmincon...
 Iter  Func-count            Fval   Feasibility   Step Length       Norm of   First-order
                                                                       step    optimality
    0           4    8.097279e+05     0.000e+00     1.000e+00     0.000e+00     5.880e+05
... [Output truncated for brevity] ...
   11         180    1.903221e+03     0.000e+00     1.299e-06     1.886e-06     3.411e+04

Local minimum possible. Constraints satisfied.

=== Optimization Results Comparison ===
Algorithm  Kp  Ki  Kd  Cost (J)
GA  0.9381  0.1535  0.2019  11.0393
PSO  0.9565  0.1522  0.1967  11.1218
fmincon  2.4508  0.0001  0.0000  1903.2208
```

Visual Verification and Step Response Analysis

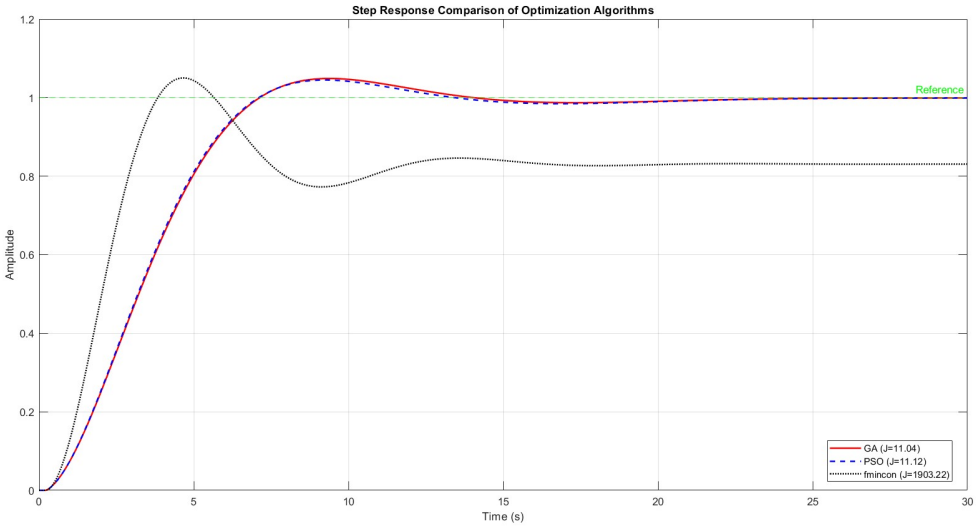


Figure 31: Closed-loop unit step response comparison. The global optimizers (GA, PSO) achieve excellent tracking, while the local optimizer (fmincon) fails dramatically.

By examining Figure 31 and the final parameter tables, several critical observations can be made regarding the system's dynamic behavior:

- **Performance of Heuristic Algorithms (GA & PSO):** Both GA and PSO converged to a highly similar region of the parameter space. Their proportional gains are near 0.95, integral gains near 0.15, and derivative gains near 0.20. Because the parameters are nearly identical, their step response curves perfectly overlap in the plot (solid blue and dashed red lines). The response exhibits an excellent rise time, minimal overshoot (well below the 5% threshold), and settles exactly to the reference value of 1.0 with zero steady-state error. This proves that both global optimizers successfully navigated the complex penalty landscape to find a globally optimal, strictly feasible solution.
- **Failure of the Local Optimizer ('fmincon'):** The gradient-based algorithm completely failed to tune the PID controller. It returned a highly aggressive proportional gain ($K_p = 2.4508$) but effectively zeroed out the integral and derivative terms ($K_i = 0.0001, K_d = 0.0000$). Consequently, the system acts as a pure proportional controller on a Type-0 delayed plant. As seen in the black dotted line in Figure 31, this results in a highly oscillatory response (due to the high K_p interacting with the transport delay) and a massive steady-state error, failing to ever reach the reference value of 1.0. The massive final cost of $J \approx 1903.22$ indicates that it heavily violated the steady-state error and overshoot penalties.

topographical Analysis of the Cost Function

The stark contrast in performance between the algorithms is not a coincidence; it is a direct consequence of how the cost function is constructed mathematically.

Our objective function utilizes discrete logical checks (e.g., ‘if overshoot ¿ 5‘) that add massive scalar constants to the cost. This creates a highly **non-convex** and **discontinuous** topographical landscape. The parameter space is filled with flat plateaus (regions where penalties are active) dropping suddenly into narrow, steep valleys (feasible regions).

‘fmincon‘ relies on calculating the Jacobian/gradient of the cost function. In a landscape defined by massive step-penalties, calculating derivatives via finite differences yields either zero (on a flat penalty plateau) or near-infinity (at the boundary of a penalty zone). As a result, the gradient approximations become meaningless, and ‘fmincon‘ immediately gets ”stuck” in a local minimum, unable to see the feasible valley located further away in the parameter space.

Conversely, GA and PSO are derivative-free, stochastic search algorithms. By scattering dozens of search agents across the entire parameter space and sharing information globally, they easily bypass local discontinuous traps and step-penalties, converging steadily into the narrow feasible region to minimize the ITAE. This experiment decisively proves that for simulation-in-the-loop control tuning with strict, discontinuous penalty constraints, global heuristic algorithms are mandatory.

```
    clc; clc; clear; close all;

lb = [0, 0, 0];
ub = [10, 10, 10];
K0 = [1, 1, 1];

%ga :
disp('Running Genetic Algorithm (ga)...');
options_ga = optimoptions('ga', 'PopulationSize', 50, ...
    'MaxGenerations', 50, 'Display', 'iter');
[K_ga, J_ga] = ga(@objectiveFunction, 3, [], [], [], [], lb, ub, [], options_ga);

%particleswarm
disp('Running Particle Swarm Optimization...');
options_pso = optimoptions('particleswarm', 'SwarmSize', 50, ...
    'MaxIterations', 50, 'Display', 'iter');
[K_pso, J_pso] = particleswarm(@objectiveFunction, 3, lb, ub, options_pso);

%sqp
disp('Running fmincon...');
options_fmincon = optimoptions('fmincon', 'Algorithm', 'sqp', 'Display', 'iter');
[K_fmincon, J_fmincon] = fmincon(@objectiveFunction, K0, [], [], [], [], lb, ub, [], optio

%%

fprintf('Algorithm\t Kp\t\t Ki\t\t Kd\t\t Cost (J)\n');
fprintf('GA\t\t %.4f\t %.4f\t %.4f\t %.4f\n', K_ga(1), K_ga(2), K_ga(3), J_ga);
fprintf('PSO\t\t %.4f\t %.4f\t %.4f\t %.4f\n', K_pso(1), K_pso(2), K_pso(3), J_pso);
fprintf('fmincon\t %.4f\t %.4f\t %.4f\t %.4f\n', K_fmincon(1), K_fmincon(2), K_fmincon(3),

%plot
```

```
s = tf('s');
G = 2 / ((5*s + 1)*(2*s + 1));
G.InputDelay = 0.2;
Tf = 0.05;

% construct controllers
C_ga = pid(K_ga(1), K_ga(2), K_ga(3), Tf);
C_pso = pid(K_pso(1), K_pso(2), K_pso(3), Tf);
C_fmincon = pid(K_fmincon(1), K_fmincon(2), K_fmincon(3), Tf);

CL_ga = feedback(C_ga * G, 1);
CL_pso = feedback(C_pso * G, 1);
CL_fmincon = feedback(C_fmincon * G, 1);

% simulate
t = 0:0.01:30;
[y_ga, ~] = step(CL_ga, t);
[y_pso, ~] = step(CL_pso, t);
[y_fmincon, ~] = step(CL_fmincon, t);

% Plot
figure;
plot(t, y_ga, 'r', 'LineWidth', 1.5); hold on;
plot(t, y_pso, 'b--', 'LineWidth', 1.5);
plot(t, y_fmincon, 'k:', 'LineWidth', 1.5);
yline(1, 'g--', 'Reference');
title('Step Response Comparison of Optimization Algorithms');
xlabel('Time (s)');
ylabel('Amplitude');
legend(sprintf('GA (J=%.2f)', J_ga), ...
       sprintf('PSO (J=%.2f)', J_pso), ...
       sprintf('fmincon (J=%.2f)', J_fmincon), 'Location', 'best');
grid on;
```

function :

```
    function J = objectiveFunction(K)

    Kp = K(1);
    Ki = K(2);
    Kd = K(3);
    Tf = 0.05;


    s = tf('s');
```

```
  G = 2 / ((5*s + 1)*(2*s + 1));
  G.InputDelay = 0.2;

  C = pid(Kp, Ki, Kd, Tf);



  CL_Y = feedback(C * G, 1);


  CL_U = feedback(C, G);


t = (0:0.1:50)';

  [y, ~] = step(CL_Y, t);
  [u, ~] = step(CL_U, t);


  error = 1 - y;
  ITAE = trapz(t, t .* abs(error));
  J = ITAE;


  penalty_weight = 10000;


  OS = max(0, max(y) - 1) * 100;
  if OS > 5
      J = J + penalty_weight * (OS - 5);
  end


  ess = abs(1 - y(end));
  if ess > 0.01
      J = J + penalty_weight * ess;
  end


  max_u = max(abs(u));
  if max_u > 5
      J = J + penalty_weight * (max_u - 5);
  end


  if any(isnan(y)) || any(isinf(y)) || y(end) > 2
      J = 1e6;
```

```
    end
end
```

Part (h): Effect of Genetic Algorithm Parameters

To evaluate how changing the core hyperparameters of the Genetic Algorithm (GA)—specifically the population size and the number of generations—affects the optimization performance and the resulting PID controller parameters.

analysis: The performance of a Genetic Algorithm relies heavily on the balance between exploration (searching the entire parameter space) and exploitation (refining good solutions).

- **Population Size (N_{pop}):** The population size determines the number of candidate PID parameter sets evaluated in each iteration.
  - *Increase:* A larger population provides a wider sampling of the search space, significantly improving exploration. This reduces the likelihood of the algorithm getting trapped in local minima (especially given the harsh penalty landscape of our objective function) and increases the probability of finding the true global optimum. However, it proportionally increases the computational time per generation.
  - *Decrease:* A smaller population speeds up computation but risks premature convergence to suboptimal PID gains, potentially failing to satisfy the strict overshoot or steady-state error constraints.
- **Generation Number (N_{gen}):** This parameter dictates the maximum number of iterations the algorithm is allowed to run.
  - *Increase:* Allowing more generations gives the GA more time to exploit and fine-tune the surviving parameter sets. It allows the cost function J (ITAE) to converge to a tighter minimum. However, beyond a certain point, the improvement becomes negligible (diminishing returns), and the extra generations merely waste computational resources.
  - *Decrease:* If the generation limit is set too low, the algorithm may terminate before the PID parameters have fully converged into the optimal feasible region, resulting in higher ITAE or constraint violations.

my conclusion: In the context of tuning this delayed plant, increasing both the population size and the generation number generally improves the reliability of finding a strictly feasible, high-performance controller with a lower cost J. However, this comes at the direct expense of computational speed. Optimal tuning requires finding the threshold where the ITAE cost stabilizes, ensuring robust convergence without excessive simulation overhead.

Part (i): Robustness and Sensitivity Analysis

To verify the robustness of the PID controller tuned via the Genetic Algorithm ($K_p = 0.9381$, $K_i = 0.1535$, $K_d = 0.2019$), a MATLAB script was developed to evaluate its performance under external disturbances and plant parameter variations.

1. Disturbance Rejection

A unit step reference was applied at $t = 0$, followed by a load disturbance of magnitude 0.5 introduced at $t = 10$ seconds. The closed-loop response to the disturbance is governed by the transfer function $T_{dist}(s) = \frac{G(s)}{1+C(s)G(s)}$. The system's behavior was simulated using MATLAB's `lsim` command.

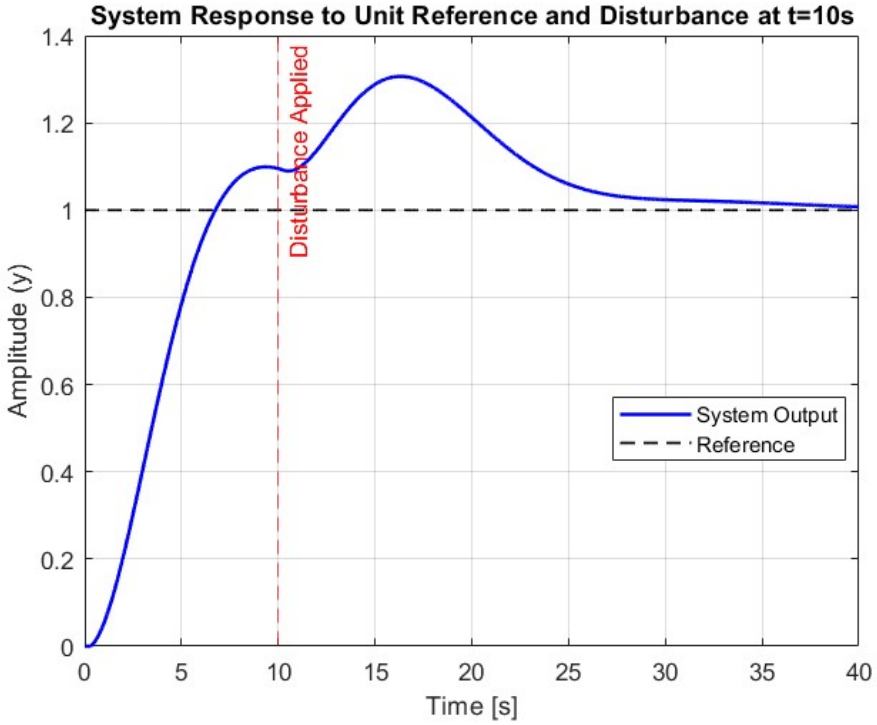


Figure 32: System response to a unit reference step and a 0.5 magnitude load disturbance at $t = 10$s.

my point of view : As shown in Figure 32, the system tracks the initial unit step reference smoothly. When the disturbance is applied at $t = 10$ seconds, the output experiences a transient deviation, peaking at approximately 1.3. However, due to the presence of integral action ($K_i > 0$) in the PID controller, the system successfully rejects the disturbance and returns to the reference setpoint ($y = 1$) with zero steady-state error.

2. Sensitivity to Parameter Variations

To assess robustness against modeling uncertainties, the plant's time constants (τ_1, τ_2) and the time delay (θ) were simultaneously varied by $\pm 20\%$. The nominal plant parameters are $\tau_1 = 4$, $\tau_2 = 3$, and $\theta = 0.3$.

- **+20% Variation (Worst-case):** Slower system dynamics and a longer dead time.
- **-20% Variation:** Faster system dynamics and a shorter dead time.

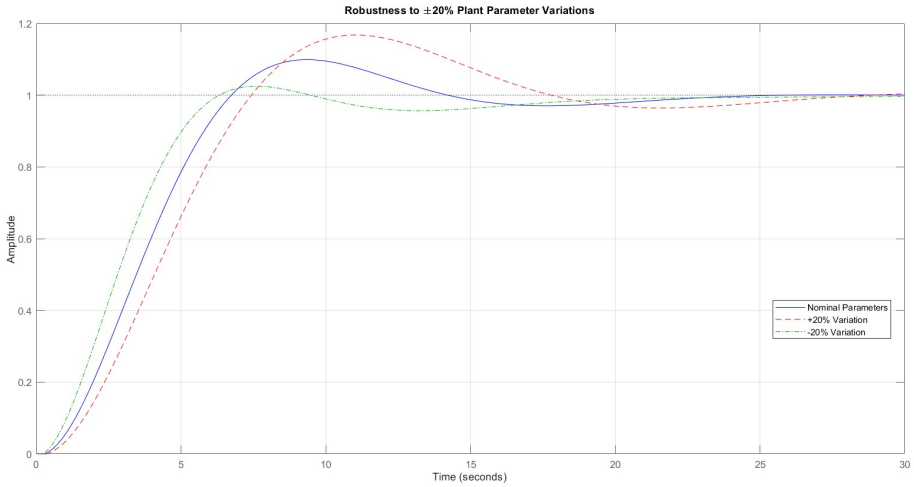


Figure 33: Step response comparison for nominal, $+20\%$, and -20% plant parameter variations.

a little discussion: Figure above demonstrates that the closed-loop system remains strictly stable despite significant parameter changes. Under the $+20\%$ variation (red dashed line), the increased delay and time constants naturally degrade performance, resulting in a higher overshoot (approx. 18%) and a longer settling time. Conversely, the -20% variation (green dash-dotted line) leads to a faster response with minimal overshoot. The fact that the controller maintains stability and acceptable tracking across this wide $\pm 20\%$ envelope confirms that the GA-optimized PID parameters are highly robust to parametric uncertainties.

12 Problem 12: PID Tuning with Derivative on Output and Set-Point Weighting

Consider the plant

$$G(s) = \frac{1}{(s+2)(8s+1)}$$

subject to disturbances, measurement noise, and parameter uncertainty.

The modified PID controller is defined as

$$u(t) = K_p\big(br(t) - y(t)\big) + K_i \int_0^t \big(r(\tau) - y(\tau)\big)\,d\tau - K_d \frac{dy(t)}{dt}$$

where b is the proportional set-point weighting parameter.

(a) Transfer Function of the Modified PID Controller

We first derive the controller expression in the Laplace domain.

Using Laplace transforms with zero initial conditions:

$$\mathscr{L}\left\{\int_0^t (r(\tau)-y(\tau))d\tau\right\} = \frac{1}{s}\big(R(s)-Y(s)\big)$$

and

$$\mathscr{L}\left\{\frac{dy(t)}{dt}\right\} = sY(s)$$

Thus,

$$U(s) = K_p\big(bR(s)-Y(s)\big) + \frac{K_i}{s}\big(R(s)-Y(s)\big) - K_d sY(s)$$

Grouping terms associated with $R(s)$ and $Y(s)$:

$$U(s) = \left(K_p b + \frac{K_i}{s}\right)R(s) - \left(K_p + \frac{K_i}{s} + K_d s\right)Y(s)$$

Define

$$C_r(s) = K_p b + \frac{K_i}{s}$$

as the controller acting on the reference signal, and

$$C_y(s) = K_p + \frac{K_i}{s} + K_d s$$

as the controller acting on the measured output.

Therefore,

$$U(s) = C_r(s)R(s) - C_y(s)Y(s)$$

Hence, the modified PID structure separates the reference and feedback paths:

$$U(s) = \left(K_p b + \frac{K_i}{s}\right)R(s) - \left(K_p + \frac{K_i}{s} + K_d s\right)Y(s)$$

The derivative term acts only on the output signal. This avoids differentiation of the reference signal and eliminates derivative kick during step changes in the set-point.

(b) Closed-Loop Transfer Functions

Assume:

- $d(t)$ is an additive input disturbance,
- $n(t)$ is additive measurement noise,

- the measured output is

$$y_m(t) = y(t) + n(t)$$

In the Laplace domain:

$$Y_m(s) = Y(s) + N(s)$$

The controller uses $Y_m(s)$, therefore:

$$U(s) = C_r(s)R(s) - C_y(s)\big(Y(s) + N(s)\big)$$

The plant input is affected by disturbance:

$$Y(s) = G(s)\big(U(s) + D(s)\big)$$

Substituting the controller equation:

$$Y(s) = G(s)\left[C_r(s)R(s) - C_y(s)\big(Y(s) + N(s)\big) + D(s)\right]$$

Expanding:

$$Y(s) = G(s)C_r(s)R(s) - G(s)C_y(s)Y(s) - G(s)C_y(s)N(s) + G(s)D(s)$$

Rearranging:

$$Y(s)\big(1 + G(s)C_y(s)\big) = G(s)C_r(s)R(s) - G(s)C_y(s)N(s) + G(s)D(s)$$

Hence,

$$Y(s) = \frac{G(s)C_r(s)}{1 + G(s)C_y(s)}R(s) + \frac{G(s)}{1 + G(s)C_y(s)}D(s) - \frac{G(s)C_y(s)}{1 + G(s)C_y(s)}N(s)$$

We now derive each requested transfer function.

1. Reference to Output

Setting $D(s) = 0$ and $N(s) = 0$:

$$\frac{Y(s)}{R(s)} = \frac{G(s)C_r(s)}{1 + G(s)C_y(s)}$$

Substituting:

$$C_r(s) = K_p b + \frac{K_i}{s}$$

and

$$C_y(s) = K_p + \frac{K_i}{s} + K_d s$$

gives

$$\frac{Y(s)}{R(s)} = \frac{G(s)\left(K_p b + \frac{K_i}{s}\right)}{1 + G(s)\left(K_p + \frac{K_i}{s} + K_d s\right)}$$

2. Reference to Control Signal

From

$$U(s) = C_r(s)R(s) - C_y(s)Y(s)$$

substitute $Y(s)/R(s)$:

$$\frac{U(s)}{R(s)} = C_r(s) - C_y(s)\frac{G(s)C_r(s)}{1+G(s)C_y(s)}$$

Factorizing:

$$\frac{U(s)}{R(s)} = \frac{C_r(s)}{1+G(s)C_y(s)}$$

Therefore,

$$\frac{U(s)}{R(s)} = \frac{K_p b + \frac{K_i}{s}}{1+G(s)\left(K_p + \frac{K_i}{s} + K_d s\right)}$$

3. Disturbance to Output

Setting $R(s) = 0$ and $N(s) = 0$:

$$\frac{Y(s)}{D(s)} = \frac{G(s)}{1+G(s)C_y(s)}$$

Thus,

$$\frac{Y(s)}{D(s)} = \frac{G(s)}{1+G(s)\left(K_p + \frac{K_i}{s} + K_d s\right)}$$

4. Measurement Noise to Output

Setting $R(s) = 0$ and $D(s) = 0$:

$$\frac{Y(s)}{N(s)} = -\frac{G(s)C_y(s)}{1+G(s)C_y(s)}$$

Hence,

$$\frac{Y(s)}{N(s)} = -\frac{G(s)\left(K_p + \frac{K_i}{s} + K_d s\right)}{1+G(s)\left(K_p + \frac{K_i}{s} + K_d s\right)}$$

5. Measurement Noise to Control Signal

Using

$$U(s) = -C_y(s)\big(Y(s) + N(s)\big)$$

for $R(s) = 0$,

substitute

$$Y(s) = -\frac{G(s)C_y(s)}{1+G(s)C_y(s)}N(s)$$

Then,

$$\frac{U(s)}{N(s)} = -C_y(s) + \frac{G(s)C_y^2(s)}{1+G(s)C_y(s)}$$

Combining terms:

$$\frac{U(s)}{N(s)} = -\frac{C_y(s)}{1+G(s)C_y(s)}$$

Therefore,

$$\frac{U(s)}{N(s)} = -\frac{K_p + \frac{K_i}{s} + K_d s}{1+G(s)\left(K_p + \frac{K_i}{s} + K_d s\right)}$$

(c) Effect of the Set-Point Weighting Parameter b

The parameter b affects only the proportional action associated with the reference signal.

Observe:

$$C_r(s) = K_p b + \frac{K_i}{s}$$

while

$$C_y(s) = K_p + \frac{K_i}{s} + K_d s$$

The disturbance rejection transfer function is

$$\frac{Y(s)}{D(s)} = \frac{G(s)}{1+G(s)C_y(s)}$$

This expression does not contain b. Therefore:

$$b \, does \, not \, affect \, disturbance \, rejection$$

However, the reference tracking transfer function is

$$\frac{Y(s)}{R(s)} = \frac{G(s)C_r(s)}{1+G(s)C_y(s)}$$

Since $C_r(s)$ contains b, the parameter directly modifies set-point tracking dynamics.

Effect of Large b If $b > 1$:

- proportional action on the set-point becomes aggressive,

- rise time decreases,
- overshoot typically increases,
- control effort becomes larger.

Effect of Small b If $0 < b < 1$:

- proportional response to set-point changes becomes softer,
- overshoot decreases,
- actuator stress decreases,
- reference tracking becomes slower.

Special Case $b = 0$ When $b = 0$:

$$C_r(s) = \frac{K_i}{s}$$

Thus, proportional action does not respond to reference changes. Only the integral term tracks the set-point.

This structure minimizes aggressive control reactions to set-point steps and is commonly used when disturbance rejection is more important than fast reference tracking.

Therefore, for pure disturbance rejection:

$$b_{optimal} = 0$$

because disturbance rejection performance remains unchanged while unnecessary proportional excitation from reference changes is eliminated.

(d) Sensitivity of the Control Signal to Measurement Noise

We compare two controller structures.

1. Standard PID with Derivative on Error

The standard PID controller is

$$u(t) = K_p e(t) + K_i \int e(t)dt + K_d \frac{de(t)}{dt}$$

where

$$e(t) = r(t) - y(t) - n(t)$$

Then,

$$\frac{de(t)}{dt} = \frac{dr(t)}{dt} - \frac{dy(t)}{dt} - \frac{dn(t)}{dt}$$

Thus the noise contribution becomes

$$-K_d \frac{dn(t)}{dt}$$

In the frequency domain:

$$sN(s)$$

Hence, high-frequency measurement noise is amplified proportionally to frequency magnitude:

$$|s| = \omega$$

As frequency increases, the derivative term amplifies noise strongly.

This causes:

- large control signal oscillations,
- actuator wear,
- chattering,
- reduced robustness.

Additionally, if the reference changes abruptly, the derivative of the error contains a large impulse-like component:

$$\frac{dr(t)}{dt}$$

which produces derivative kick.

2. Modified PID with Derivative on Output

The modified controller uses

$$-K_d \frac{dy(t)}{dt}$$

instead of

$$K_d \frac{de(t)}{dt}$$

Therefore, the derivative term does not differentiate the reference signal.

Consequently:

- derivative kick is eliminated,
- actuator stress is reduced,
- set-point changes produce smoother control action.

The measurement noise still enters through the feedback path, but only through the measured output signal. The absence of differentiated set-point signals significantly improves practical behavior.

Thus, compared to the standard PID:

Derivative on output provides lower practical sensitivity to measurement noise and eliminates derivative kick

Therefore, the modified PID structure is generally preferred in industrial control systems, especially in processes with noisy sensors and abrupt set-point changes.

(e) Derivative Filtering in PID Controllers

In practical control systems, the derivative term of a PID controller is highly sensitive to high-frequency measurement noise. Since differentiation amplifies rapid signal variations, even small sensor noise can produce large oscillations in the control signal.

To reduce this problem, practical PID controllers do not use an ideal derivative term:

$$K_d s$$

Instead, a filtered derivative term is implemented:

$$D(s) = \frac{K_d s}{1 + \tau s}$$

where:

- K_d is the derivative gain,
- τ is the derivative filter time constant.

The denominator introduces a first-order low-pass filter which attenuates high-frequency components before differentiation becomes excessive.

Physical Interpretation The ideal derivative operator:

$$K_d s$$

has gain magnitude

$$|K_d j\omega| = K_d \omega$$

Thus, as frequency increases:

$$\omega \to \infty$$

the gain grows without bound.

This means high-frequency measurement noise is strongly amplified.

By introducing the filter:

$$\frac{1}{1 + \tau s}$$

the derivative action becomes frequency-limited.

At low frequencies:

$$\tau s \ll 1$$

therefore:

$$D(s) \approx K_d s$$

and normal derivative behavior is preserved.

At high frequencies:

$$\tau s \gg 1$$

then:

$$D(s) \approx \frac{K_d}{\tau}$$

which is bounded.

Hence, the filter prevents infinite amplification of noise.

Modified PID Transfer Function

The modified PID controller with:

- set-point weighting,
- derivative on output,
- derivative filtering

is written as

$$u(t) = K_p\big(br(t) - y(t)\big) + K_i \int_0^t (r(\tau) - y(\tau))d\tau - filteredderivativeterm$$

Replacing the ideal derivative with the filtered derivative:

$$D(s) = \frac{K_d s}{1 + \tau s}$$

the Laplace-domain controller becomes

$$U(s) = \left(K_p b + \frac{K_i}{s}\right) R(s) - \left(K_p + \frac{K_i}{s} + \frac{K_d s}{1 + \tau s}\right) Y(s)$$

Define the modified feedback controller:

$$C_y(s) = K_p + \frac{K_i}{s} + \frac{K_d s}{1 + \tau s}$$

and the reference path controller:

$$C_r(s) = K_p b + \frac{K_i}{s}$$

Therefore, the complete filtered PID controller structure is

$$U(s) = C_r(s)R(s) - C_y(s)Y(s)$$

or explicitly,

$$U(s) = \left(K_p b + \frac{K_i}{s}\right) R(s) - \left(K_p + \frac{K_i}{s} + \frac{K_d s}{1+\tau s}\right) Y(s)$$

Closed-Loop Transfer Function with Derivative Filtering

Using the plant:

$$G(s) = \frac{1}{(s+2)(8s+1)}$$

the closed-loop reference-to-output transfer function becomes

$$\frac{Y(s)}{R(s)} = \frac{G(s)\left(K_p b + \frac{K_i}{s}\right)}{1 + G(s)\left(K_p + \frac{K_i}{s} + \frac{K_d s}{1+\tau s}\right)}$$

Similarly, the disturbance and noise transfer functions are modified by replacing the ideal derivative term with:

$$\frac{K_d s}{1+\tau s}$$

throughout the loop transfer function.

Effect of τ on Noise Sensitivity

The parameter τ determines the strength of the filtering effect.

Small τ If τ is very small:

$$\tau \to 0$$

then:

$$\frac{K_d s}{1+\tau s} \approx K_d s$$

The controller behaves almost like an ideal derivative controller.

Advantages:

- fast transient response,
- strong damping,
- improved anticipation of rapid changes.

Disadvantages:

- high sensitivity to measurement noise,
- noisy control signal,
- actuator chattering.

Large τ If τ is increased, the low-pass filtering becomes stronger.
Advantages:

- reduced amplification of high-frequency noise,
- smoother control action,
- improved actuator longevity,
- better robustness in noisy environments.

Disadvantages:

- weaker derivative action,
- slower transient response,
- reduced damping,
- possible increase in overshoot and settling time.

Thus, τ creates a trade-off between:

$$noise attenuation \quad and \quad dynamic performance$$

Frequency-Domain Interpretation

Consider the magnitude of the filtered derivative term:

$$\left| \frac{K_d j\omega}{1 + j\omega\tau} \right| = \frac{K_d \omega}{\sqrt{1 + \omega^2 \tau^2}}$$

For low frequencies:

$$\omega \ll \frac{1}{\tau}$$

the magnitude behaves approximately as:

$$K_d \omega$$

For high frequencies:

$$\omega \gg \frac{1}{\tau}$$

the magnitude approaches:

$$\frac{K_d}{\tau}$$

Therefore, the filter limits high-frequency gain and prevents excessive noise amplification.

Practical Design Considerations

In industrial PID tuning, the filter time constant is usually selected as a small fraction of the derivative time constant:

$$\tau \approx \frac{T_d}{10} \quad to \quad \frac{T_d}{20}$$

where:

$$T_d = \frac{K_d}{K_p}$$

This provides a compromise between:

- preserving useful derivative action,
- suppressing measurement noise,
- maintaining closed-loop robustness.

Consequently, filtered derivative action is considered essential in practical PID implementations, especially in systems with noisy sensors or high-frequency disturbances.